



ALMA MATER STUDIORUM UNIVERSITY OF BOLOGNA
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING – DISI

Beyond Next-Token Training & Prediction

**Reasoning, Neuro-Symbolic AI, Large Action Models, Tools,
Autonomous Agents & Model Context Protocol**

Prof. Gianluca Moro
Tutor: Dr. Giacomo Frisoni, Lorenzo Molfetta

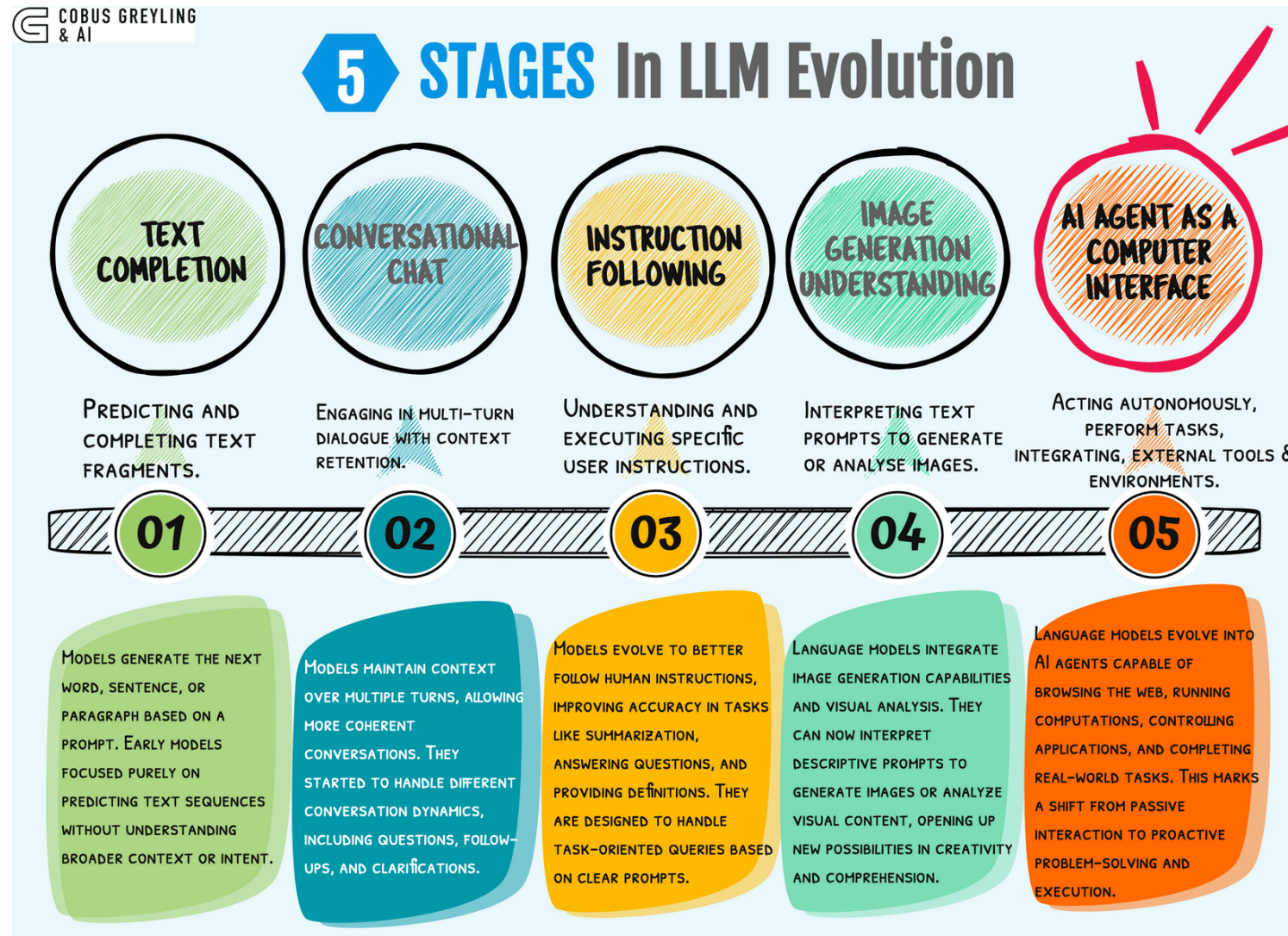
DISI, University of Bologna, Cesena
Via dell'Università, 50 I-47522 Cesena (FC), Italy
{gianluca.moro, giacomo.frisoni, lorenzo.molfetta}@unibo.it

2025: the year of agents

- Ilya Sutskeve (OpenAI co-founder), talk at NeurIPS 2024
 - Test of time award for the Seq2Seq paper
- Scaling LLMs at the pre-training stage got plateaued
 - The compute is scaling but data is not: we have but one Internet
 - Pre-training as we know it will end
- What next ? **AI systems that reason, act, and adapt**
 1. **Agents and tools on top of LLMs will fuel the progress**
 2. **Reasoning**, not intuition, will define advanced AI—complex, multi-step thinking; the more a system reasons, the more unpredictable it becomes
 3. **Understand** things from limited data
 4. **Self-awareness**

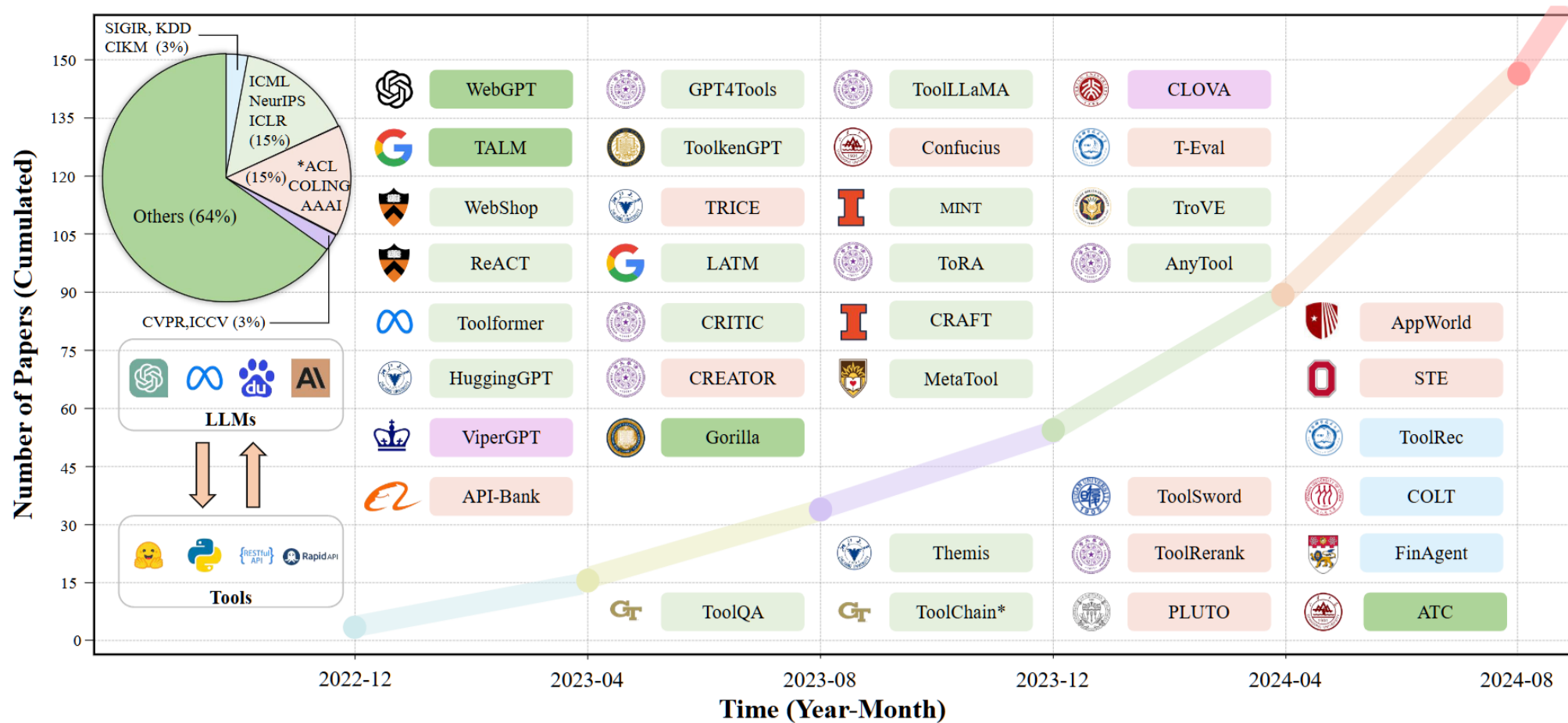


Agents are considered the next step in LLM evolution



The tool learning landscape

- Surge of tool learning concurrent with the rise of LLMs [Qu et al., 2024]

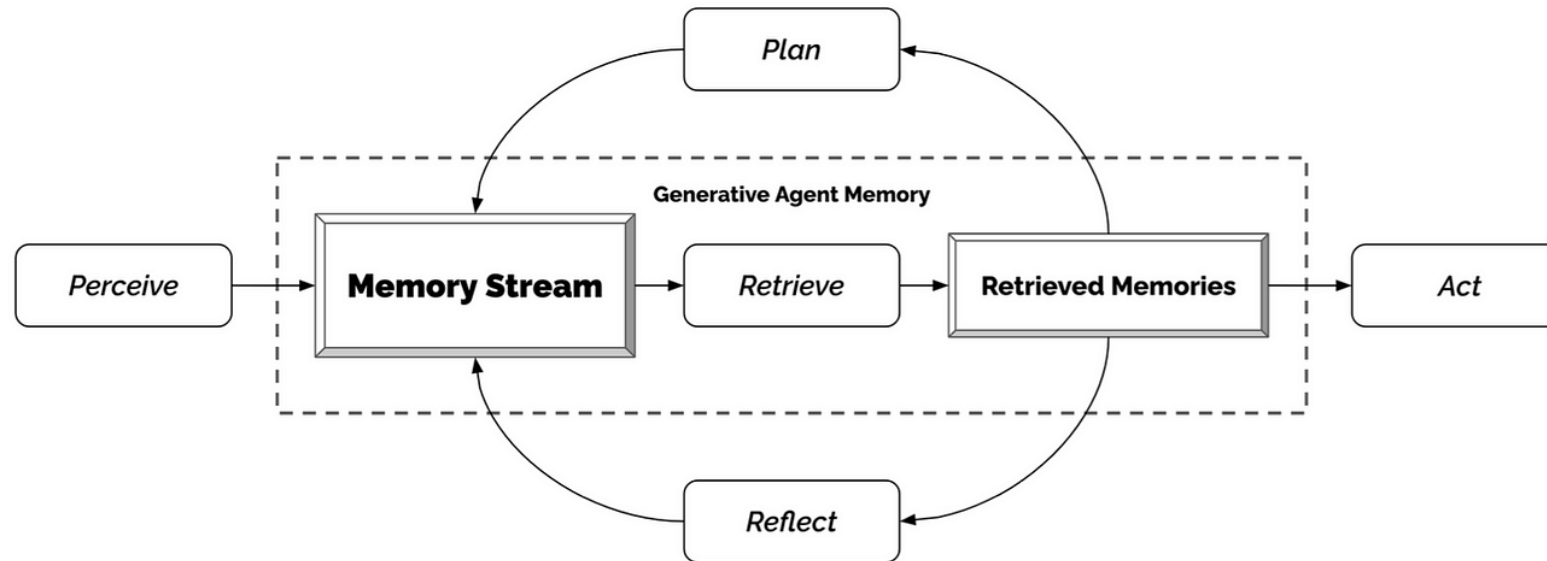


What is an autonomus agent ?



An autonomous agent is a **system** situated within and a part of an **environment** that **senses** that environment and **acts** on it, over time, in pursuit of its **own agenda** and so as to effect what it senses in the **future**.

- Franklin and Graesser (1997)



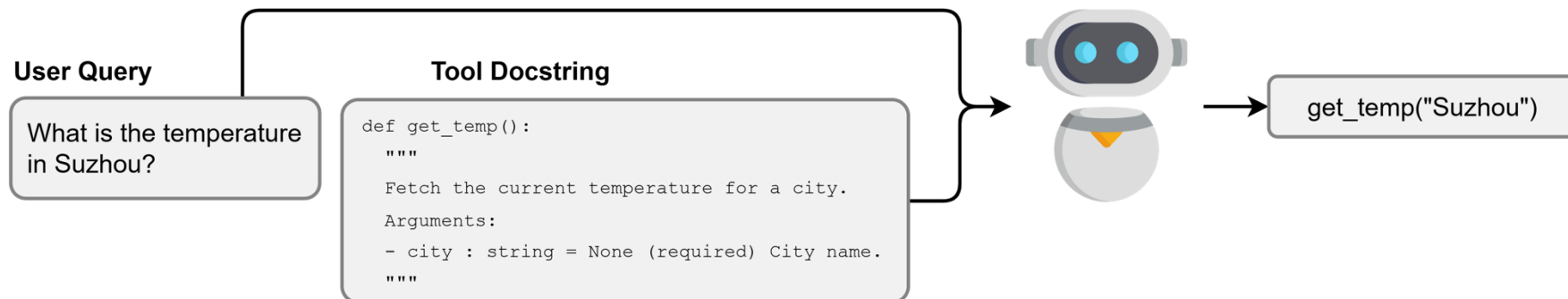
Agentic hype

- Agents are the **new shiny AI** toy after chatbots
- What is fascinating about 2025 is how people have started exploring the benefits of combining agents with LLMs
 - Language to gather knowledge, make decisions, plan next steps, learn from outcomes
 - The difference today is the engine, not the car; instead of rigid, rule-based systems, we have LLMs that are fluid and adaptable



Agentic hype

- Vanilla LLMs
 - Knowledge frozen to training data + no interaction with the outside world
- Tool-augmented LLMs (**Large Action Models – LAM**)
 - Can determine **when** and **how** to utilize external tools, expanding their functional scope
 - Access **up-to-date information**, **run** computations, **use** third-party services, etc.
 - **Unlock** far broader use cases than the model's built-in knowledge alone
- Tool usage = Generation of a function call statement  LAM = LLM + text parsing?



Foundation models

Multimodal context understanding



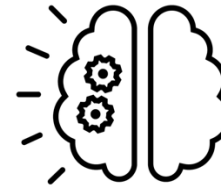
Foundation models are able to interpret context from various modalities (simultaneously)

Tool usage



Enable the usage of tools and controlling systems, e.g., via REST-APIs

Reasoning capabilities



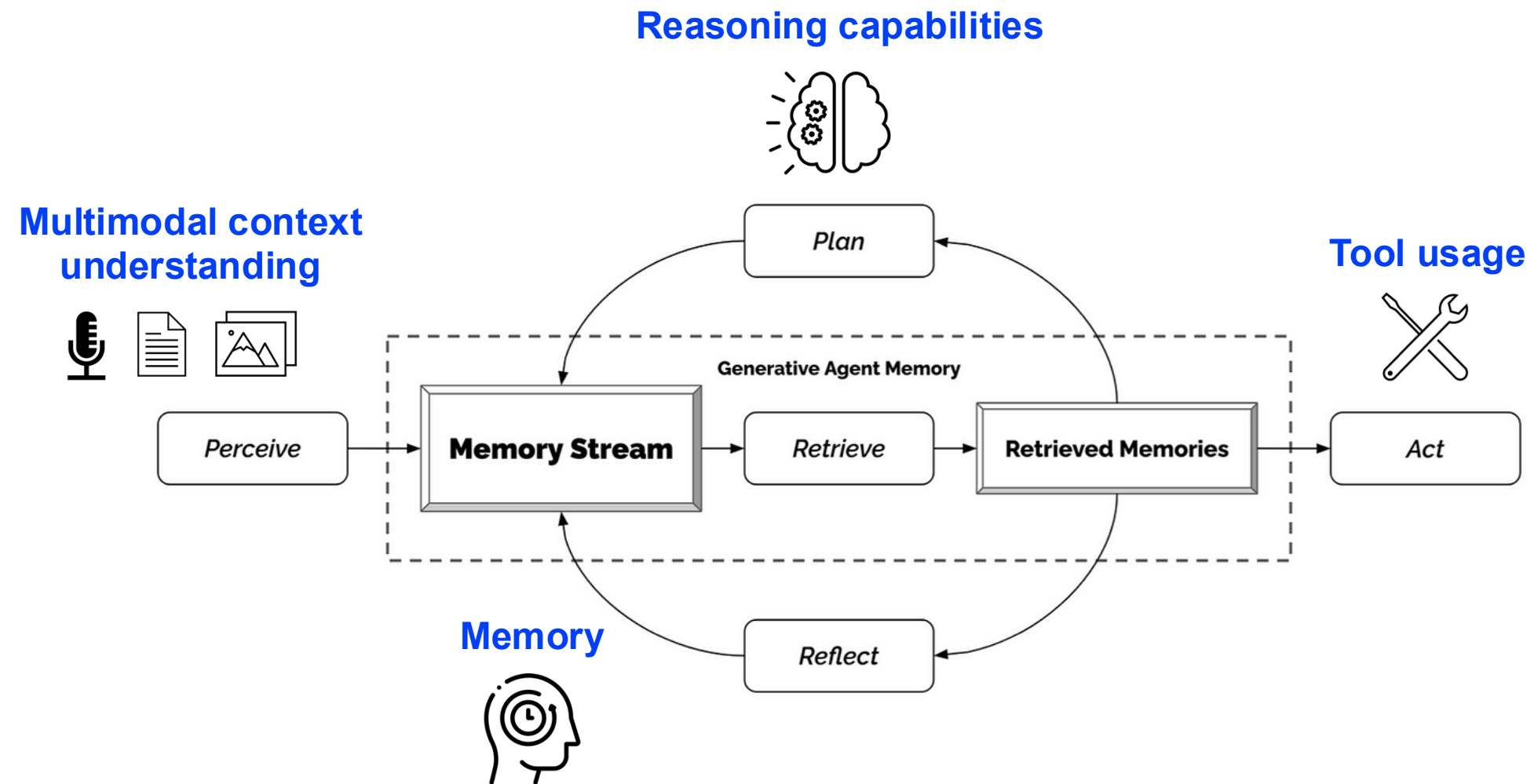
Enable planning, breaking down tasks into subtasks, self-critique and self-reflection

Memory



Storing and retrieving information, e.g., via RAG

Putting skills together



LangChain: state of AI agents

- LangChain surveyed 1,300+ developers and business leaders in September 2024 on AI agent adoption, use cases, challenges, and more
- Agent adoption trends
 - 51% of respondents are using agents in production today
 - 78% have active plans to implement agents into production soon
- Leading agent use cases
 - Research and summarization (58%)
 - Personal productivity or assistance (53.5%)
- Performance quality is the biggest barrier that holds companies back from using AI agents in production



[Full report](#)

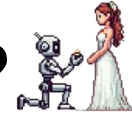
Outline

- **The illusion of Reasoning: Pitfalls of “vanilla” LLMs** 
- The Architecture of Autonomy: ReACT
- Large Action Models and Tool Calling
- Model Context Protocol
- Examples of Application

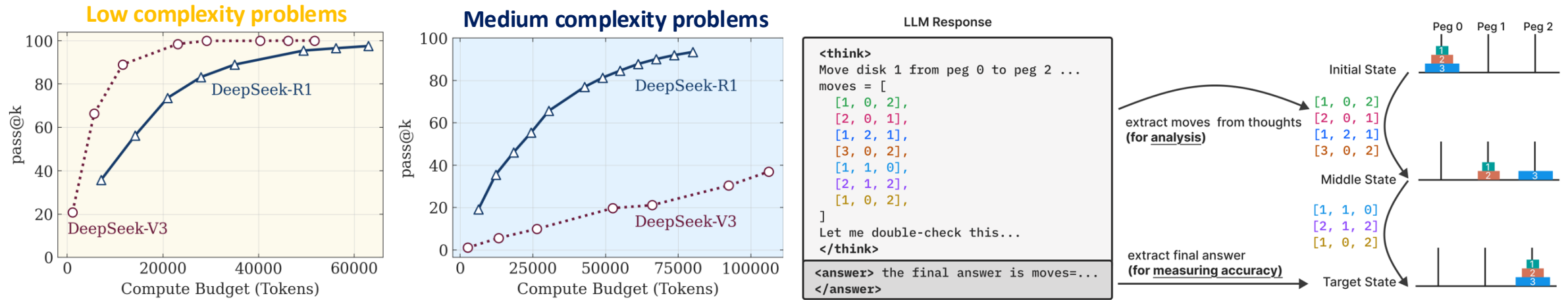
Reasoning LLMs: The Illusion of Thinking

- 2025 has **also** been the year of **reasoning** for Large Language Models
- Since the introduction of  **deepseek-R1**, all major AI companies have started training and releasing LLMs capable of producing thought chains
 - Before answering, the model produces a sequence of **goal-free** and eventually **unrelated** reasoning steps toward the final solution
 - Latest reasoning LLMs: **Alibaba QwQ and Qwen3**, **OpenAI GPT-Oss and o{1,3,4}**, **Anthropic Claude Opus/Sonnet 4.{1,5}**
- ✓ Higher **probability** of solving complex tasks, also providing *explanations*
- ✗ **Low efficiency**: lengthy answers and a high number of tokens rule the **accuracy-cost** tradeoff
- ⚠ Beware of the **snowballing effect !!**

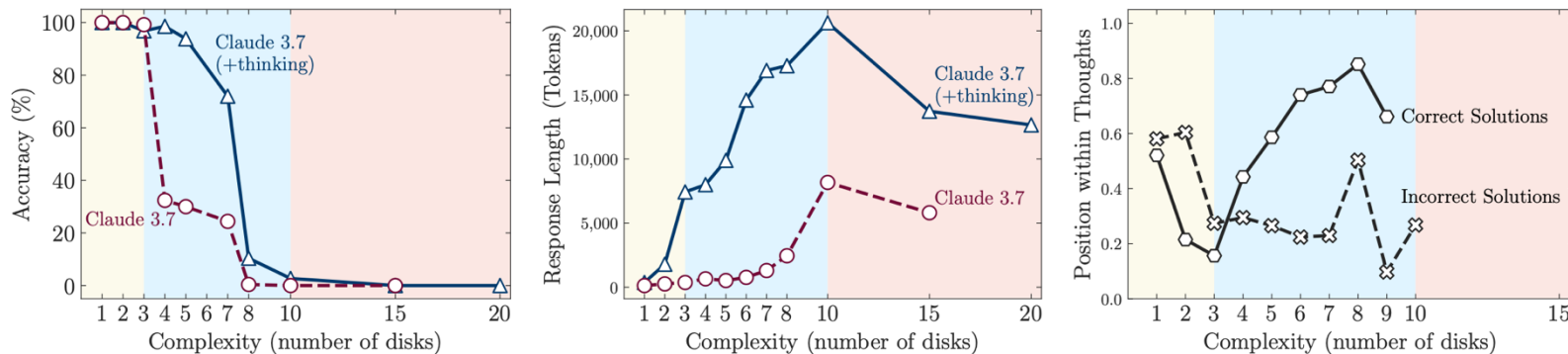
Are LLMs good only at *planning* weddings?



- In LLMs context, planning is not an emergent behavior resulting from reasoning
 - Higher thinking capabilities **counterintuitively** do not correlate with problem difficulty



DeepSeek-V3 no-reasoning; DeepSeek-R1 reasoning



[The Illusion of Thinking, Apple 2025](#)

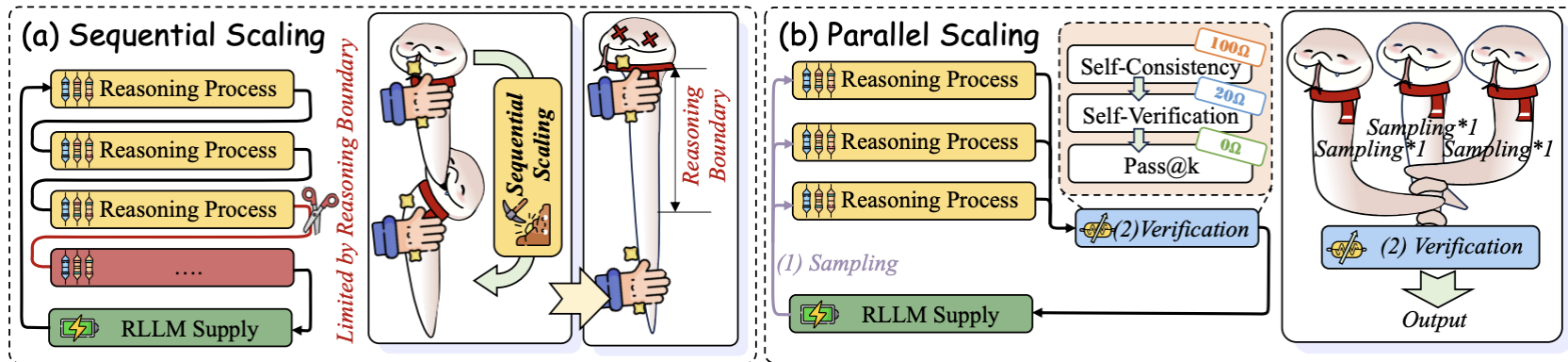
Accuracy vs. Complexity Analysis

At low complexity levels, non-thinking models tend to be more accurate and use tokens more efficiently.

As complexity rises, reasoning models perform best but need more tokens, until both methods fail beyond a threshold.

A new (old) trend: Test-time scaling

- As we seek to optimize LLM reasoning for **planning** of agentic actions, we are assisting a **shift** from training/fine-tuning to **test-time scaling**
- Test-time scaling can be classified into two approaches:
 - **Internal**: involves **adjusting model parameters** through additional training on specifically designed reasoning tasks (e.g., GRPO finetuning)
 - **External**: boost inference costs by introducing **additional computational steps**, such as re-sampling or re-generating model outputs multiple times (e.g., Self Consistency)



Test-time scaling strategies

(a) Extension of the CoT length constrained by the input boundaries of a Reasoning LLM (RLLM)

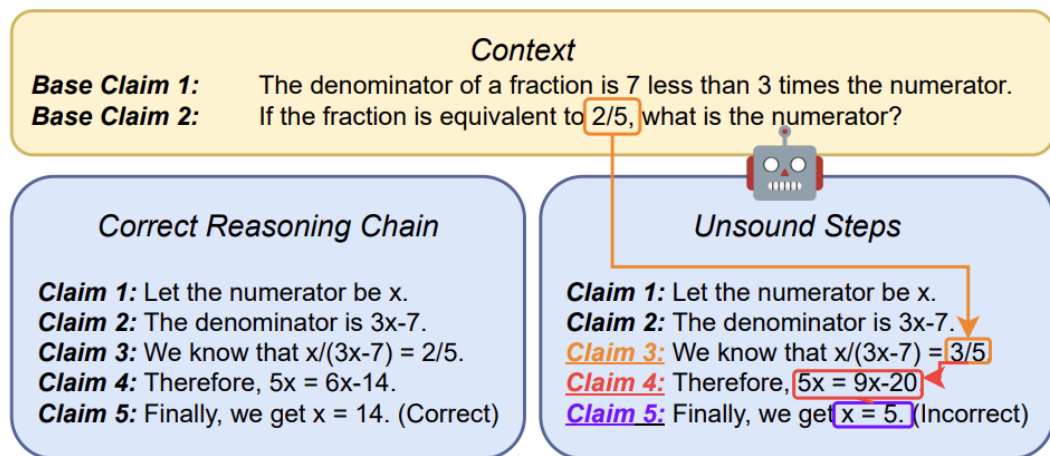
(b) Increase the sample size and aggregate multiple outcomes

For more insights on test-time scaling, look at [B. Brown et al., 2024](#), [W. You et al., 2025](#); [Z. Gan et al., 2025](#); [Q. Chen et al., 2025](#)



Snowballing thoughts: the chain of unpredictability

- During test-time inference, a single initial error in a reasoning step is carried forward and **used as a foundational premise** for all subsequent steps
- The **propagated errors** can cause later steps to be judged as logically sound by the model, but undermine the reliability of the conclusion
 - In sequential scaling, a **flawed intermediate premise** can *hardly* be recovered (next slide)



Ungrounded step

Incorrect with respect to the context

Invalid step

Logically incorrect derivation

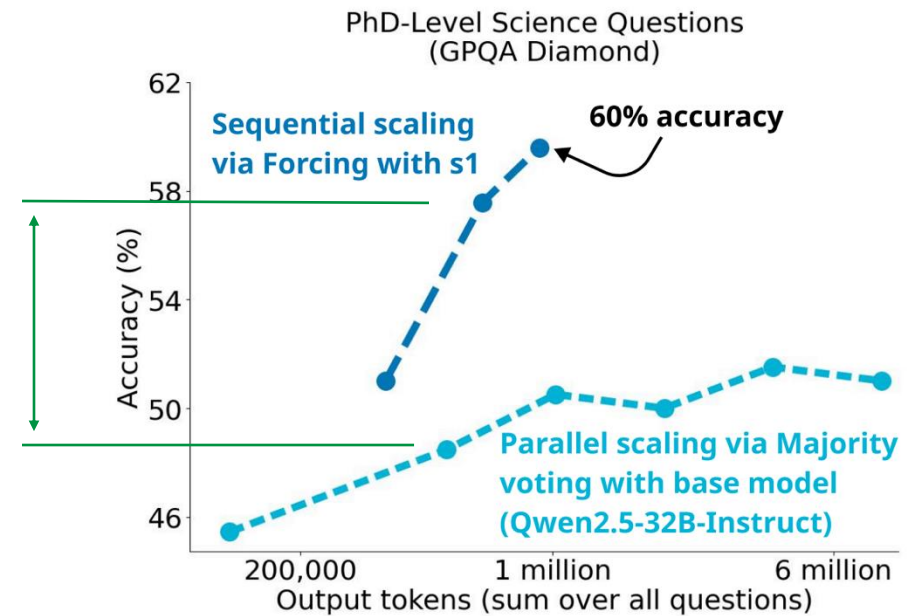
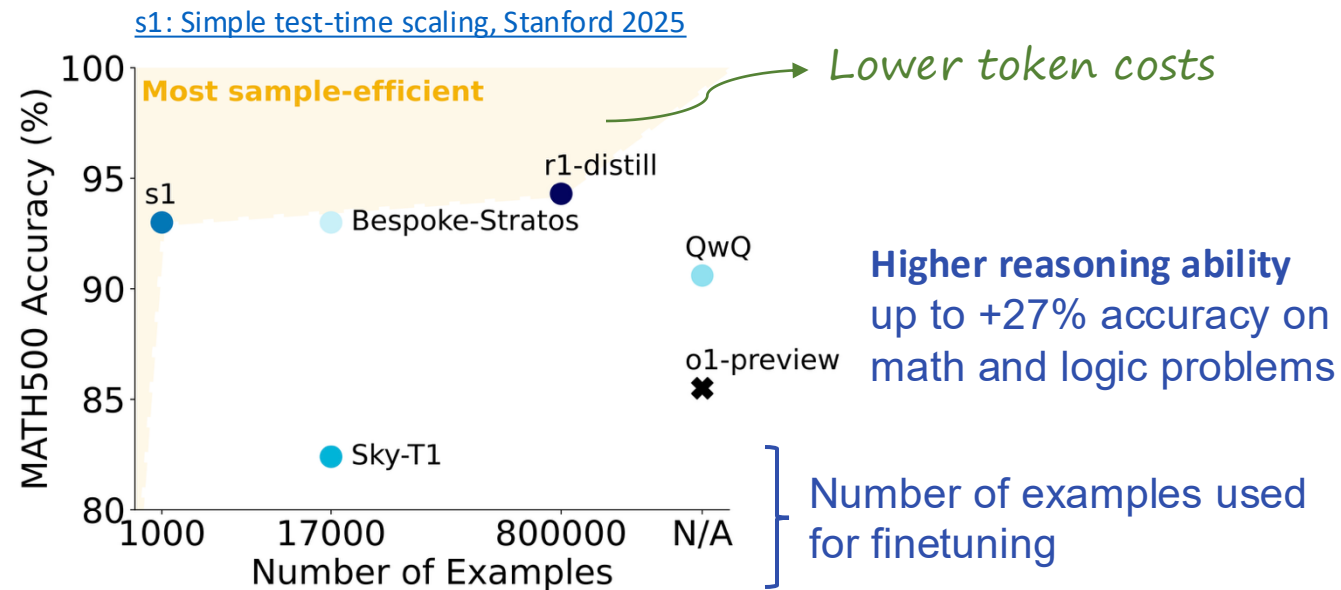
Error propagation

Logically consistent continuation of previous errors

The snowball effect is a major **concern**, especially for an **agentic planner** framework, where **choosing the wrong action step** can compromise the entire resolution process (e.g., checking the wrong year in stock market analysis)

Oh wait... How to encourage exploration

- Test-time scaling can be incentivized with some simple yet highly effective tricks
 - Prompting the model to interleave its reasoning with “**wait**” encourages it to verify its answers, often correcting faulty thinking steps



Can these “**reasoning**” traces be considered **symbolic** and **explainable** clues?

Archetype of the Neuro-Symbolic Paradigm

- Solve a **proxy** task (e.g., align embedding spaces) either before producing the final output or as **part** of the **learning strategy**
- The model and the proxy task are **tightly coupled** with a **formal language**, for example, by enforcing rules, generating code, or mining logical clauses
- The proxy task does **not** require **separate supervision** or a pretraining phase, but it is optimized into the end-to-end learning process

✓ **Higher formal grounding** capabilities over specific rules or knowledge bases

- we can derive interpretable logic rules and formal system evolution behaviors

✗ **Lower semantic power** derived from hard-constrained I/O formalisms

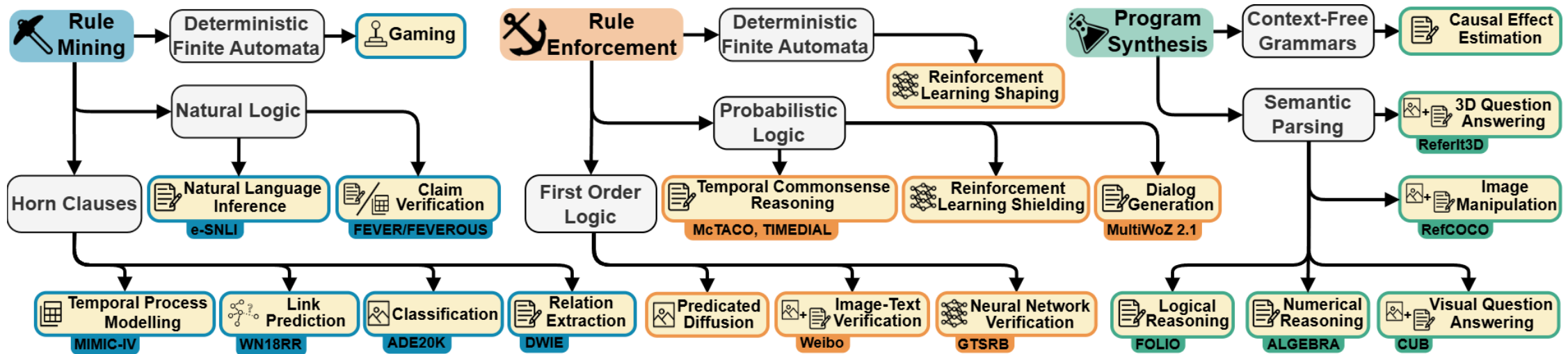
- may miss semantic nuances due to misaligned ontologies

Neuro-Symbolic Artificial Intelligence

A Task-Directed Survey in the Black-Box Models Era

Start with your **task of interest** and explore **nearby** approaches from the same **methodological family** to get insights you can adapt to your scenario

Bottom-Up



Top-Down

Deepen the **categorization** of high-level general techniques, distinguishing between different **logical formalisms** and **learning paradigms**

Program Synthesis: a hub for LLM-NeSy collaboration

- LLMs are best suited for **rich semantic** and **formalized** natural language tasks
 - **Paradigm shift:** LLMs as **probabilistic syntactic oracles**, rather than direct synthesizers, that can be exploited to guide symbolic search (don't replace it)

Auto-formalization: Translating informal mathematical and logical problems into formal specifications for theorem provers and verifiers

- **Bridge** between natural language understanding and symbolic reasoning
- Enables large-scale **synthetic data generation** for formal domains

Iterative refinement loops: LLM generation coupled with symbolic verification in feedback cycles

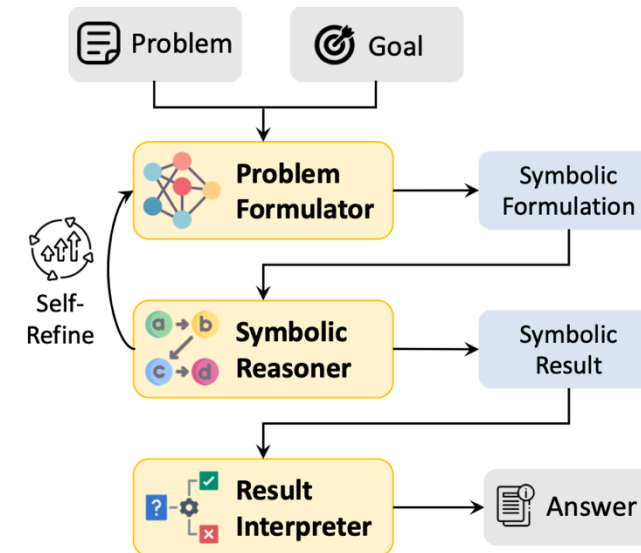
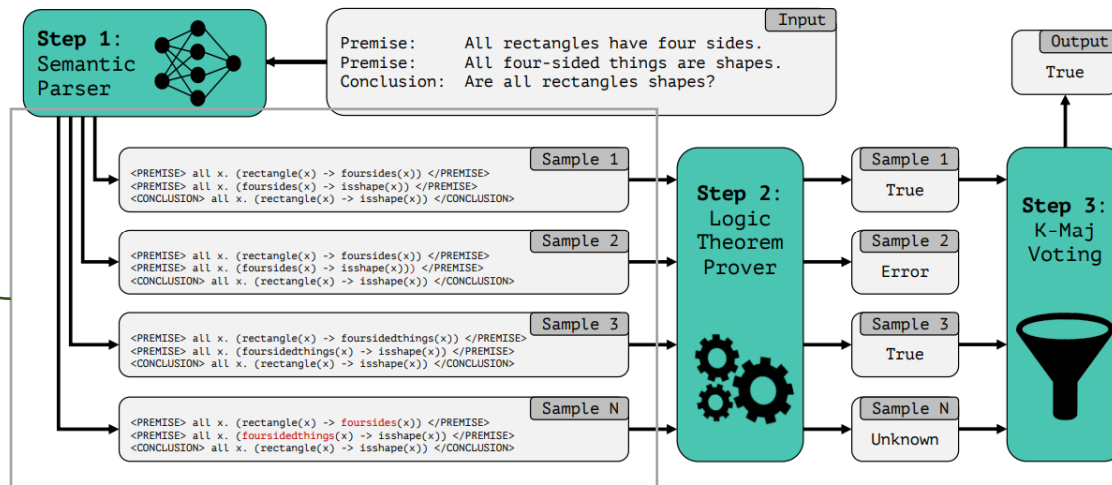
- Generate **candidate** solutions, verify with formal **tools**, and repair with counter-examples
- Probabilistic **guidance:** LLM weights search grammars to prioritize promising program regions

Program Synthesis: a matter of intermediate representations

Generate **structured programs** in either general-purpose programming or domain-specific **languages** to **guide** the system's prediction **behavior**

- Program Synthesis introduces higher **complexity**, as generating a valid program often **depends** on **semantic parsing** or **search algorithms**
 - Its **greater expressive power** distinguishes it from formal rule-based methods, which are typically constrained by the removal of quantifiers

Generate candidate symbolic derivations



- 1 LLM as a parser**
Generate a structured intermediate data
- 2 Symbolic verifier**
Process result with a symbolic program
- 3 LLM as a solver**
Generate final answer based on previous data

Outline

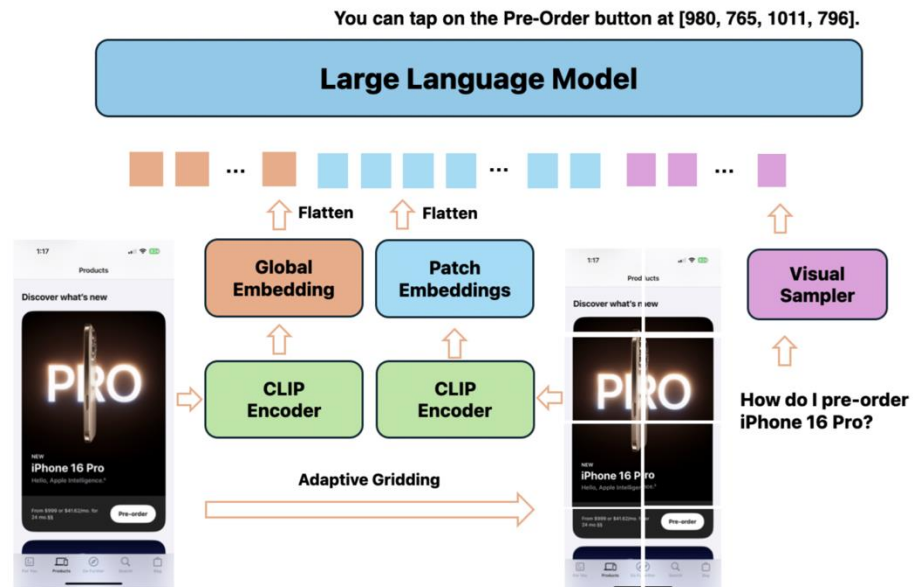
- The illusion of Reasoning: Pitfalls of “vanilla” LLMs
- **The Architecture of Autonomy: ReACT** 
- Large Action Models and Tool Calling
- Model Context Protocol
- Examples of Application

Requirements for the Architecture of Autonomy

- **Grounding:** Autonomous agents must overcome LLM reliance on **internal knowledge** alone
 - Require a mechanism to interface with **external tools** and environments to retrieve factual, up-to-date information
- **Dynamic Planning:** The pipeline must be capable of generating, tracking, and **flexibly adjusting** multi-step action plans
 - Efficiently **handle exceptions** and **recover** from initial errors without the snowballing effect
- **Trustworthiness:** The agent's decision-making process must be transparent and **human-aligned**
 - Users need to easily inspect the trajectory, distinguishing the model's **internal thought process** from external observations

Perception module

- Modality-specific encoders and multimodal models can serve as **agent sensors**
 - Understand input from various modalities, e.g., images, audio, video, text, sensors



Ferret-UI 2 @ Apple [Li et al., 2024]

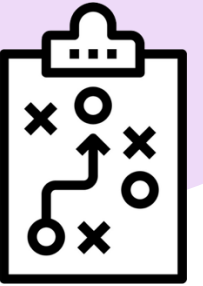
Visual encoder + decoder-only language model for universal UI understanding across iOS platforms

Input: Image	Input: Image + SoM
	
Conversation	Conversation + SoM
User What is on the left side of the right laptop?	User What is on the left side of the right laptop?
GPT-4V On the left side of the right laptop, there is a cup or mug . ❌	GPT-4V On the left side of the right laptop (numbered as 9), there is a lamp, which is numbered as 12 . ✅
User I want to find a seat close to windows, where can I sit?	User I want to find a seat close to windows, where can I sit?
GPT-4V You can sit on either of the two black chairs in front of the white desks, as they are closest to the windowed wall. ❌	GPT-4V You can sit on the chair numbered as 7 . It's the closest seat to the windows (represented by the area numbered as 1). ✅

Set-of-Mark Prompting [Yang et al., 2024]

Simply overlaying ids on image regions unleashes visual grounding and corrects answers for GPT-4V

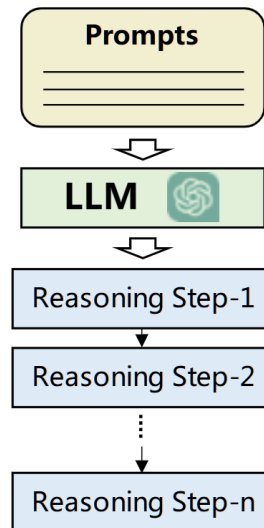
Planning module



- When faced with a complex task, humans tend to deconstruct it into simpler subtasks and solve them individually
 - 💡 Utilize the reasoning capabilities of LLMs to **create a plan and break a task down into subtasks and actionable steps**
- **Planning without feedback:** the agents do not receive feedback that can influence its future behaviors after taking actions
 - In many real-world scenarios, generating a flawless plan directly from the beginning is extremely difficult; as a result, **simply following the initial plan often leads to failure**
 - The initial plan may prove to be **non-executable** due to unpredictable transition dynamics
- **Planning with feedback:** the agent can receive feedback after taking actions
 - When examining how humans tackle complex tasks, we find that **individuals may iteratively make and revise their plans based on external feedback**
 - The feedback can be obtained from **environments** (e.g., game completion signals), **humans** (e.g., preferences), **pretrained models** (e.g., self-judgments)

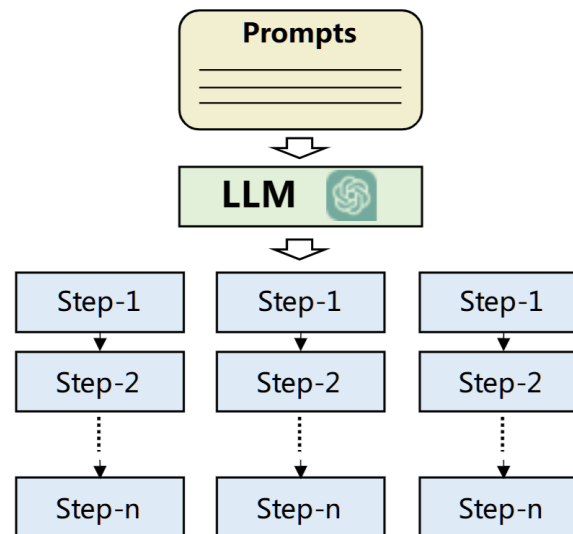
Planning module: no feedback

Single-path reasoning

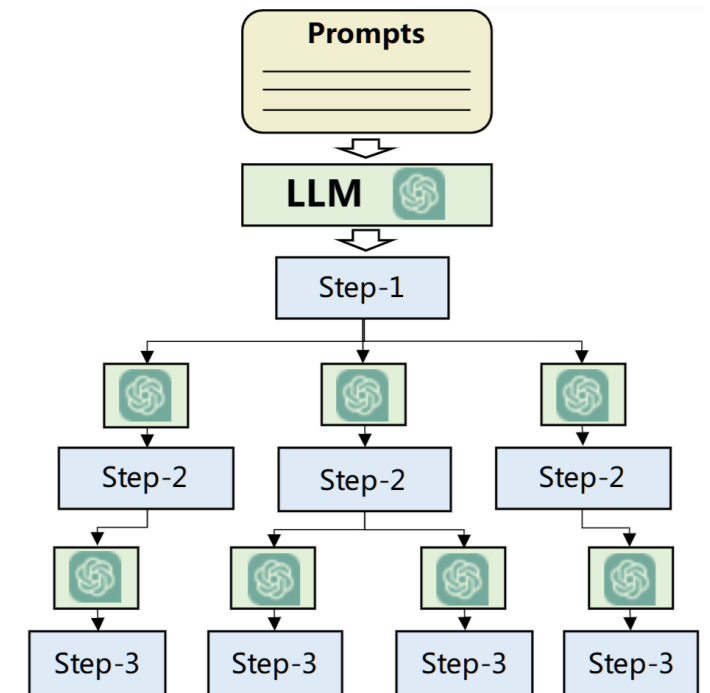


- **Chain-of-Thought (CoT)** [Kojima et al., 2022] prompts LLMs with trigger sentences like «**think step by step**»
- Reasoning steps can be incorporated as examples in the prompt [Wei et al., 2022]

Multi-path reasoning



- **Self-consistency with CoT (CoT-SC)** [Wang et al., 2022] first generates various reasoning paths and corresponding answers with CoT, then chooses the final output through majority voting



- **Tree-of-Thoughts (ToT)** [Yao et al., 2023] queries the LLM at each reasoning step to generate k thought branches for the next step

Reason **then** Act: toward schematic planning

- Introduced by [S. Yao et al., 2023](#), ReAct (**Reason+Act**) prompts the LLM to **dynamically interleave verbal reasoning traces** and task-specific **actions**
 - Combine the high-level planning of Chain-of-Thought with the ability to **interact with the external environment** (e.g., APIs, run programs, call other LLMs)

Reasoning for Action - Thought: The model's Thought step serves as an **internal monitor** to induce, track, and update the action plan explicitly

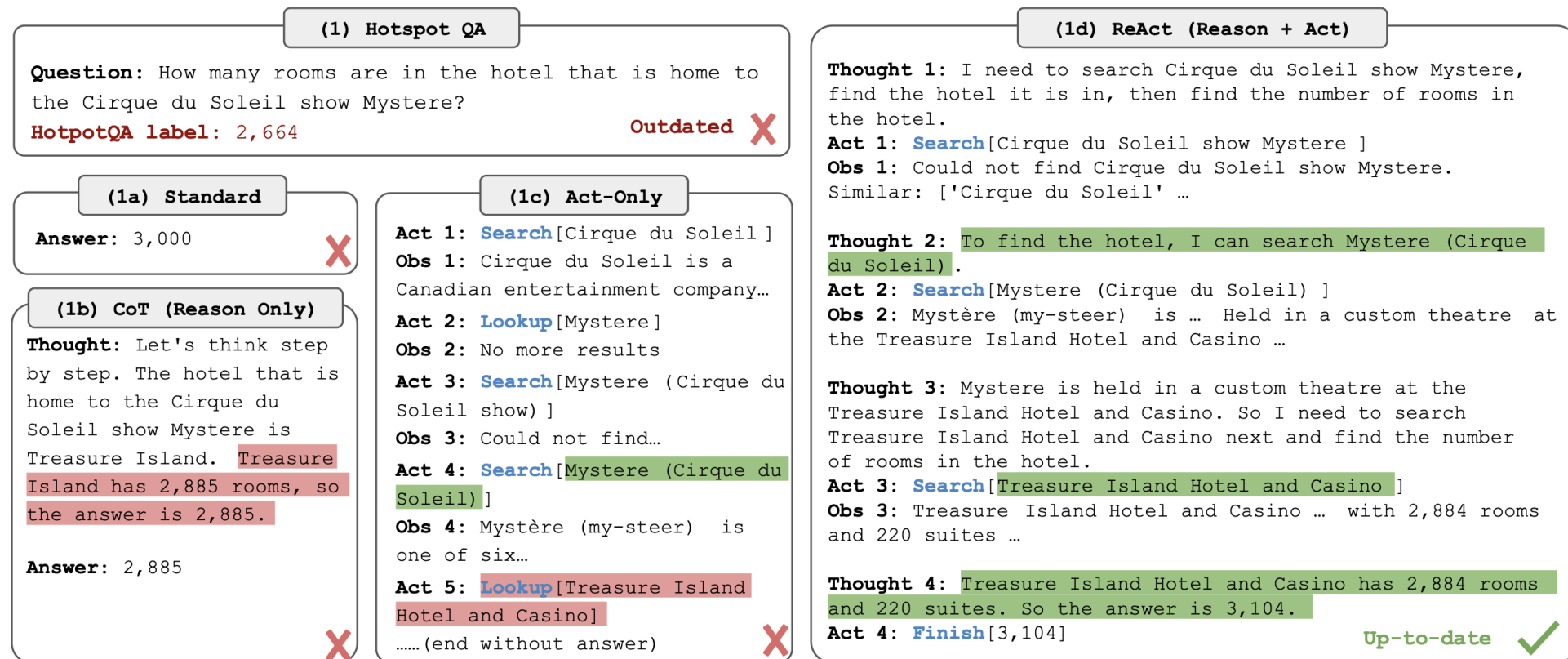
- Critical for **decomposing complex goals** into sub-tasks and injecting necessary commonsense knowledge.

Action for Reasoning - Observation: The Action allows the model to interface with external tools (e.g., search APIs) to gather information

- The resulting Observation provides real-time, **factual feedback** to ground the next reasoning step and correct its internal state

Interleaved reasoning: the best way to plan

- ReAct can **dynamically self-correct** by using the observation of a failed action to reformulate its subsequent internal plan
 - Act-Only methods fail due to a lack of **strategic planning**, while CoT methods suffer from **hallucination** due to being ungrounded



Improving ReAct: Self-Refine

- Methods like ReAct and CoT primarily focus on *planning* steps, often leading to **suboptimal initial drafts** for generation tasks (e.g., writing or complex coding)
 - Lack of an **internal, iterative quality check** to assess and improve the produced output
- **Self-Refine** ([A. Madaan et al., 2023](#)) introduces a simple loop **Generate** → **Self-Critique** → **Refine**, using the LLM itself to act as a **post-hoc editor**
 - This achieves substantial gains by focusing on **output quality convergence**, independent of the initial generation strategy

Code optimization: x, y_t

```
Generate sum of 1, ..., N
def sum(n):
    res = 0
    for i in range(n+1):
        res += i
    return res
```

FEEDBACK fb

```
This code is slow as
it uses brute force.
A better approach is
to use the formula
... (n(n+1))/2.
```

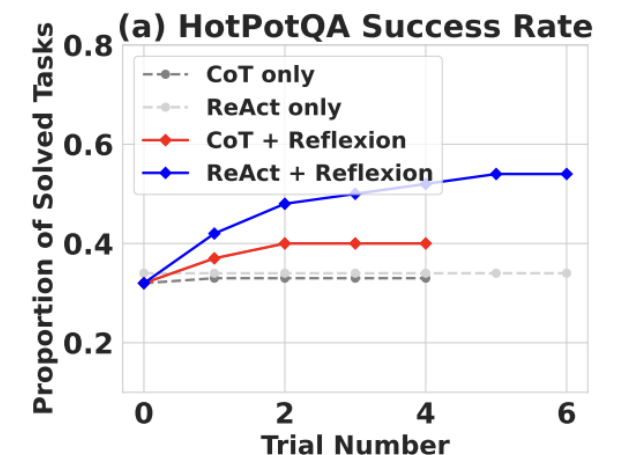
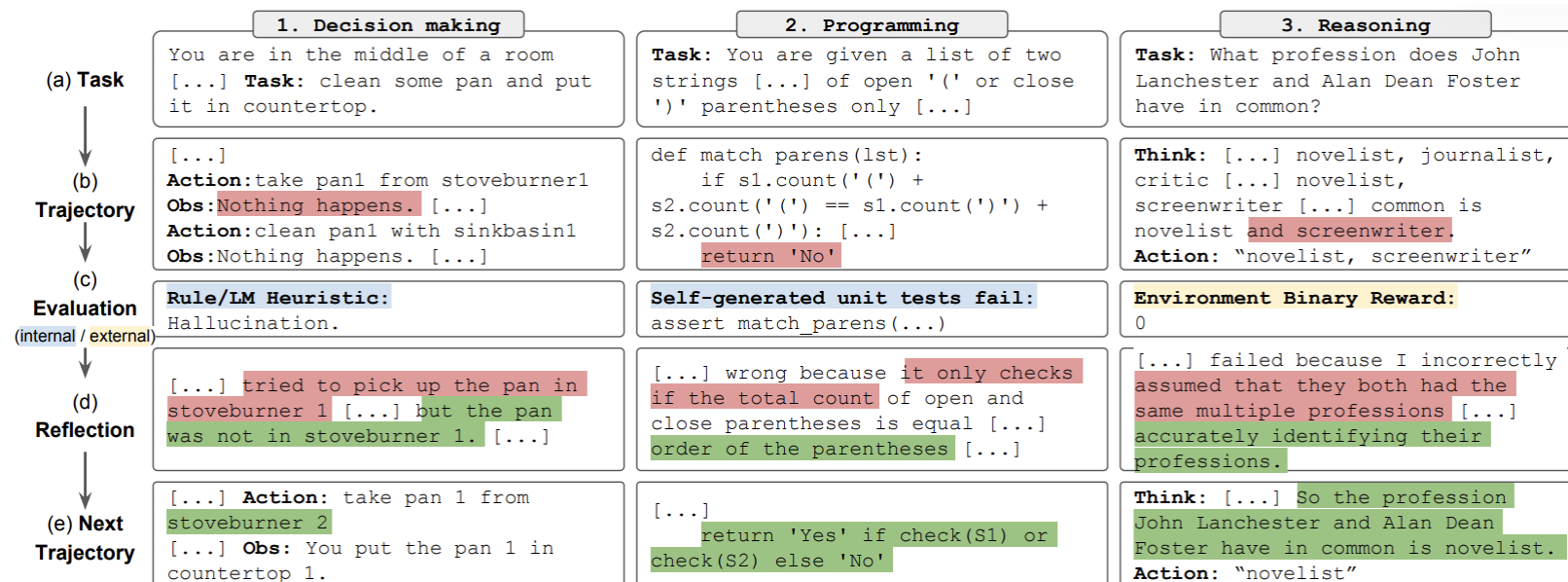
REFINE y_{t+1}

```
Code (refined)

def sum_faster(n):
    return (n*(n+1))//2
```

Improving ReAct: Reflexion

- While ReAct uses **observations** to correct a single step within the *current episode*, it has **no persistent, distilled memory** of past *failed episodes*
- Reflexion** ([N. Shinn et al., 2023](#)) converts task failures into a **linguistic reflection**, which is stored in an **Experience (long-term memory)** buffer
 - Before a new episode, the agent insert **failed reflections** from the memory into the prompt, providing a **verbal reinforcement signal** to correct the *high-level strategy*



Outline

- The illusion of Reasoning: Pitfalls of “vanilla” LLMs
- The Architecture of Autonomy: ReACT
- **Large Action Models and Tool Calling** 
- Model Context Protocol
- Examples of Application

Tools growing as toolips in April

- Growing scale, both in research and industry
 - Future LLM agents are expected to choose from **thousands** of available tools
 - E.g., ToolBench [[Qin et al., 2024](#)] already lists 10K+ tools to reflect realistic conditions
 - Industry standards are promoting the emergence of public and private tools for the most diverse tasks, I/O modalities, and domains
 - New tools emerge continuously and user needs evolve: the ideal pool is **dynamic**
- In short, LLMs are envisioned to interface with an **open-ended library of tools**
 - Similar to how a human user has access to an entire app store
 - **Tool hubs** are emerging, as we saw for models, adapters, and datasets

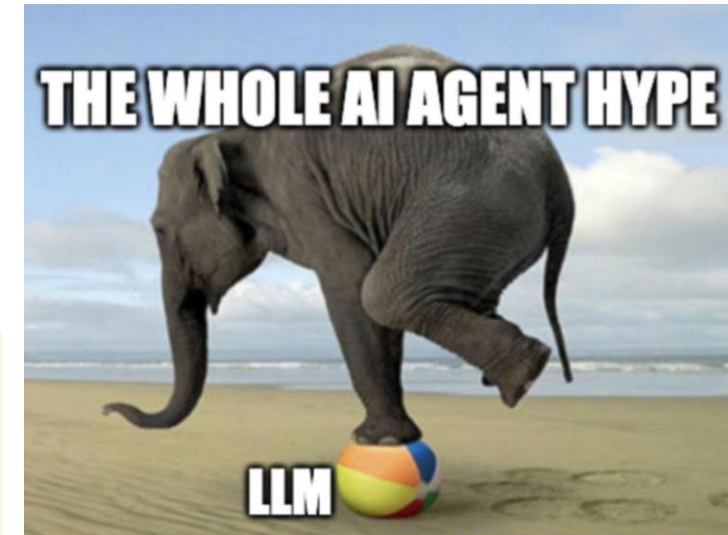


The dark side of LAM-based Agents: *prompts* (again)

- Modern agentic frameworks function as **Orchestration Pipelines**, where the LLM acts as a central executive coordinating tasks via text generation
- Under the hood, single- or multi-agent frameworks are **strongly dependent** on a well-written **system prompt**

What makes you a LAM agent...

- 1 **Linear Brain:** decompose a complex goal into a linear sequence of atomic LLM calls – the output of step t becomes the input for step $t+1$
 - Involves stochastic generation of the “next best step”
- 2 **Social Brain:** facilitates structured dialogues between collaborative agents to solve problems
 - Replaces monolithic control with emergent problem-solving, where an agent’s reply triggers another agent’s function
- 3 **Short-Term Memory:** orchestrator dynamically routes information into and out of the context window, retrieving the right context at the right time



The need for a clear rubric

- LAMs require **clear instructions** to define their boundaries regarding planning workflow, environment interaction, and answer format delivery
 - Most of the time, LAMs are not trained but instructed to act as agents

You are a research assistant who answers questions through **iterative reasoning and research**.

Process

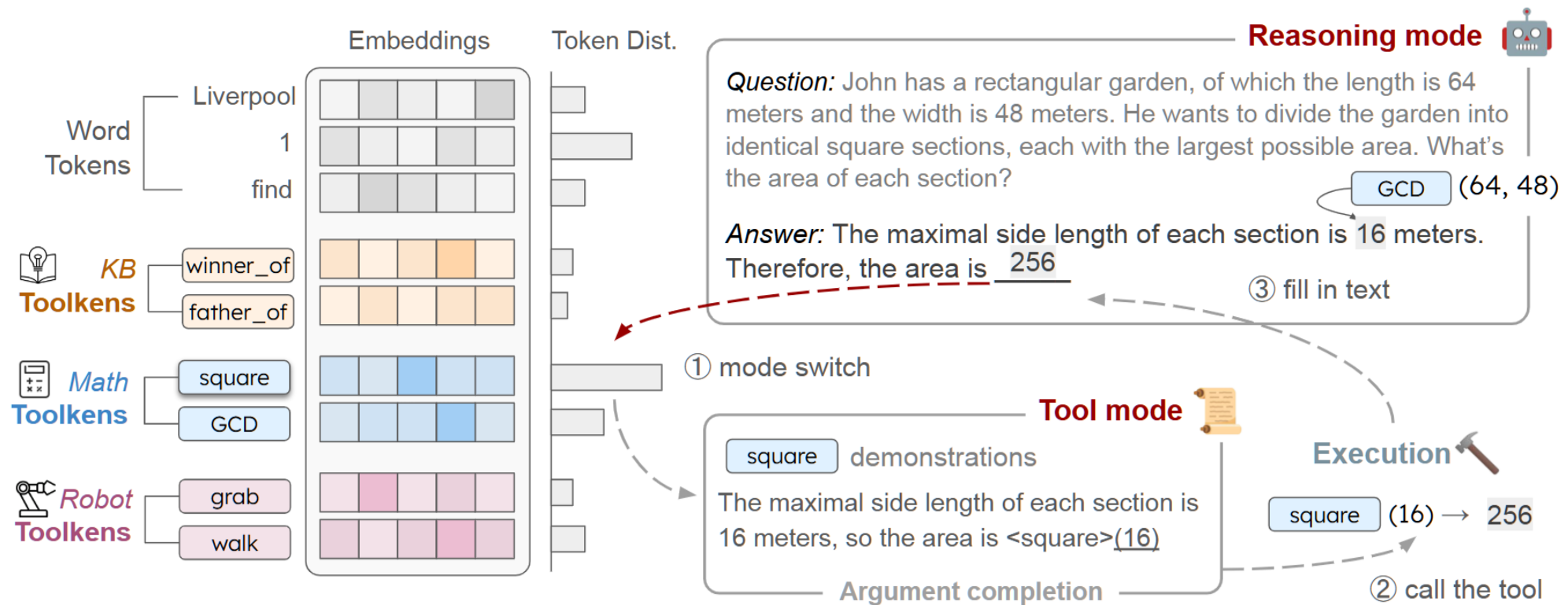
- Use `<think></think>` tags to show your reasoning at any point.
- Use `<call_tool name=\"...\">query</call_tool>` when you need information (see tools below).
- You can alternate between thinking and searching multiple times.
- Only provide `<answer></answer>` tags when you have enough information for a complete response. If the problem asks for a specific, short-form answer, you can also put the answer string in the `\boxed{}` format.
- **Support** every non-trivial claim with retrieved evidence. Wrap the exact claim span in `<cite id=\"ID1,ID2\">`



Do not even try using *base* models for agentic purposes!!

Tool as a token: an exception against current trends

- ToolkenGPT ([S. Hao et al., 2024](#)) represents each tool as a special **toolken**
 - Once a toolken is predicted, (1) the LLM switch to the tool mode, which provides a few demonstrations of the same tool to complete the arguments, (2) the tool call is executed, (3) the result is sent back to the text to continue the problem resolution



Action module: build the action set with tools

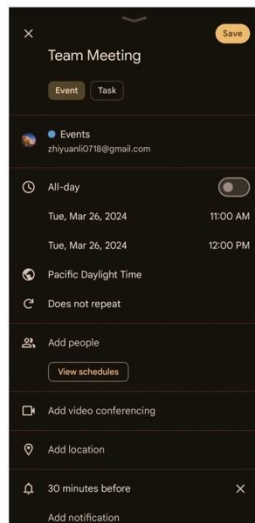
- Module responsible for **translating the agent's decisions into specific outcomes**
- Equipping LLMs with the capability to dynamically interact with **external tools**
 - Tools = functions or APIs
 - This integration not only **improves the problem-solving potential of LLMs**, but also dramatically **expands their functional scope**
 - For example, tools can enable LLMs use a calculator, set calendar events, access real-time weather information, run queries on a database, make web searches on Google
- **Tool selection** ( → )
 - Choosing through a retriever or directly allowing LLMs to pick from a provided list of tools
 - When there are many tools, a tool retriever is typically used to identify the top- K candidates
- **Tool calling** ()
 - Extracting the required parameters from the user query in accordance with the specifications outlined in the tool description

Action module: tool calling examples

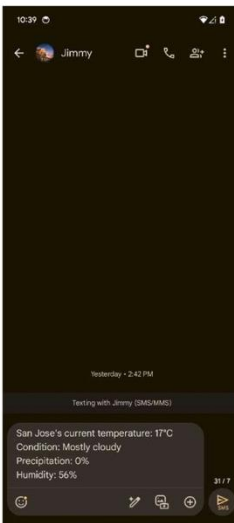
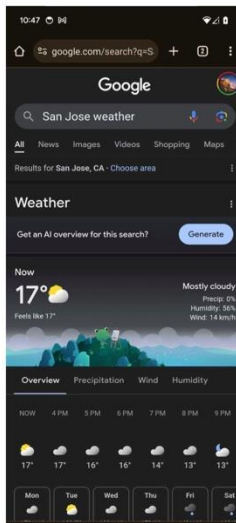
- Most of the time, tools are presented to the LLM as python/json class definitions
 - Gorilla dataset by UC Berkeley and Microsoft [Patil et al., 2023]



Create **calendar** reminder for Team Meeting on 2024 March 26 11am to 12pm



Tell me **weather** today in San Jose and send **text message** to Jimmy about the weather information



Search **Youtube** for Taylor Swift Concert



```
def take_a_photo(camera="back", resolution="1080p"):
    """
    Captures a photo using the specified camera and resolution settings.

    Parameters:
    - camera (str, optional): Specifies the camera to use. Can be 'front'
      or 'back'. The default is 'back'. Optional to provide.
    - resolution (str, optional): Sets the photo resolution. Options
      include '720p', '1080p', and '4K'. The default is '1080p'. Optional to
      provide.

    Returns:
    - str: The string contains the file path of the captured photo if
      successful, or an error message if not. Example: '/storage/emulated/0/
      Pictures/MyApp/IMG_20240310_123456.jpg'
    """
```

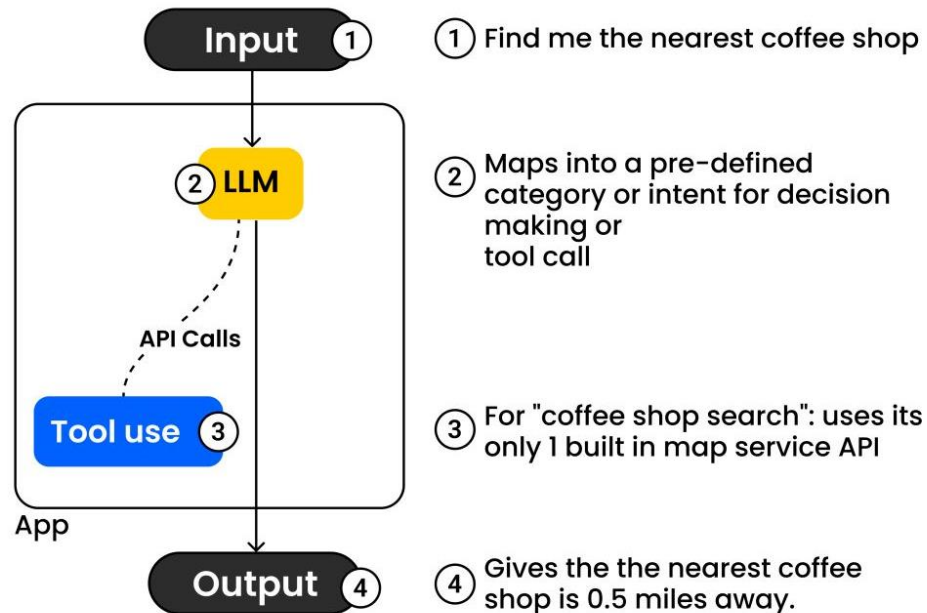
Generation:

```
take_a_photo(camera="front", resolution="720p")
```

Action module: tools vs agents

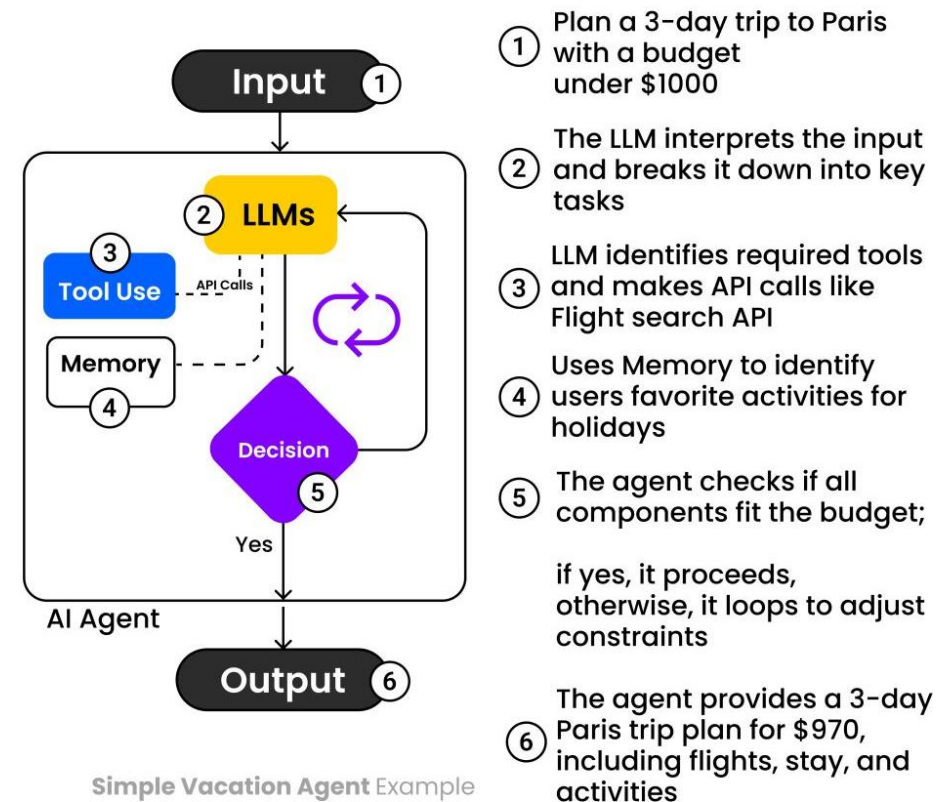
- Agents require independence and goal-driven behavior

This is **not** an AI Agent



Simple Query bot example

This is what AI Agent Actually is



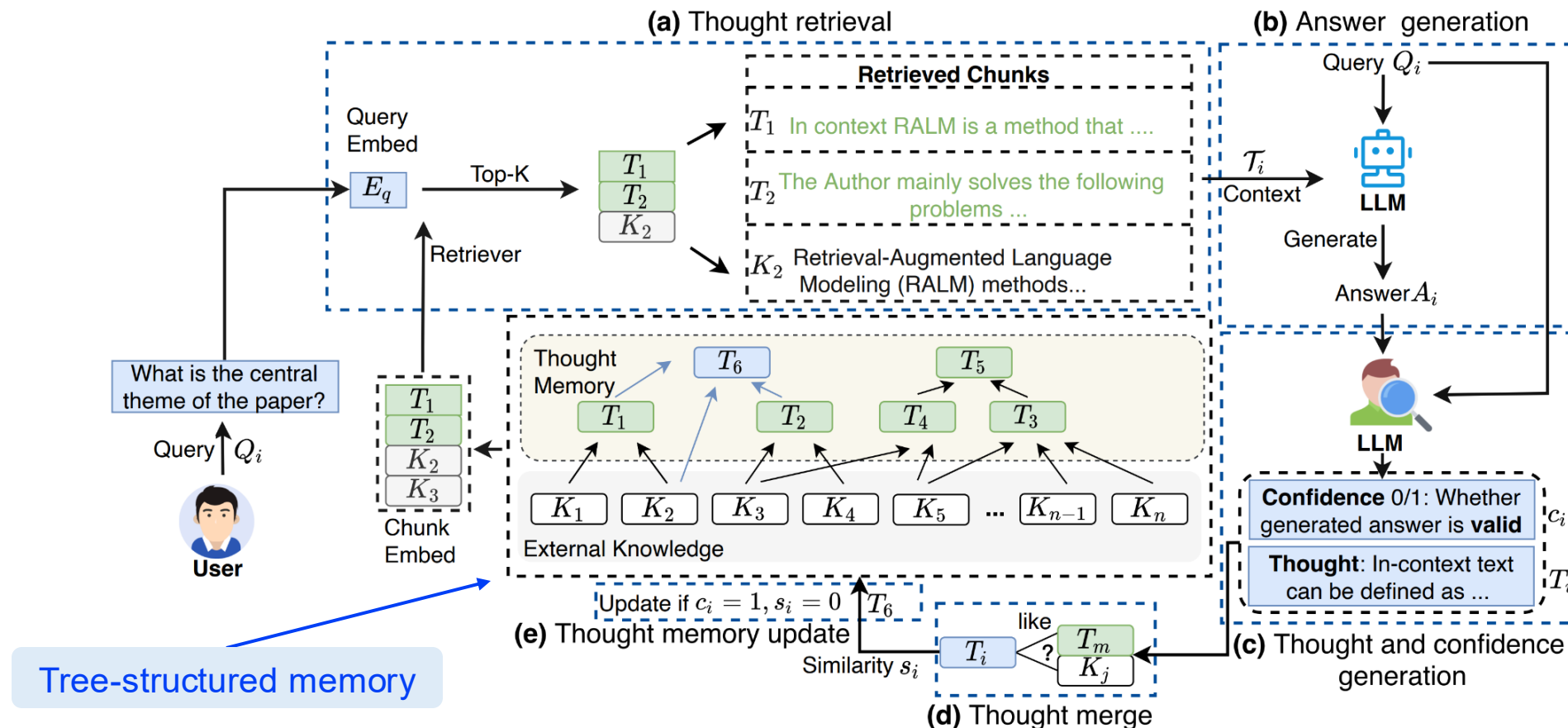
Simple Vacation Agent Example

Memory module

- Stores information perceived from the environment and leverages the recorded memories to facilitate future actions
 - It can help the agent to accumulate experiences, self-evolve, and behave in a more consistent, reasonable, and effective manner
- Memory structure
 - **Unified memory**
 - Only simulates the human short-term memory, which is usually realized by in-context learning, i.e., the memory information is directly written into the prompts
 - However, the limited context window of LLMs restricts incorporation of comprehensive memories into prompts, which can impair agent performance
 - **Hybrid memory**
 - Short-term memory to temporarily buffer recent perceptions + long-term memory to consolidate important information over time, frequently using vector databases
- Operations: reading (e.g., approximate nearest neighbor), writing (duplicates and storage limit), reflection (summarize and infer more abstract information)

Memory module: thought retriever [Feng et al., 2024]

- An LLM-agnostic self-evolving retrieval framework that leverages historical LLM responses to handle new queries (per-instance behavior update w/o training)

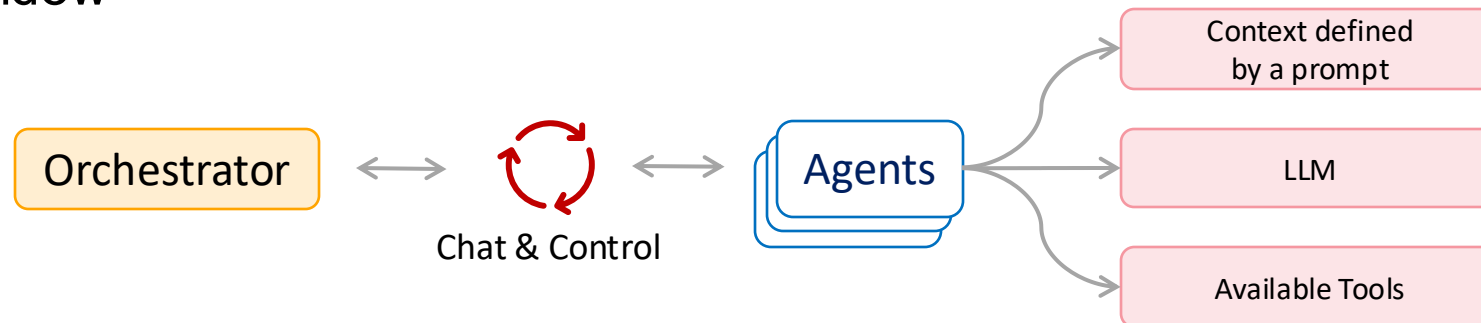


Multi-Agent Systems (MAS)

- **Limitations** of single-agent systems
 - Too many tools at disposal
 - Context grows too complex to keep track of
 - Need for multiple specialization areas (e.g., planner, researcher, math expert)
- Researchers and developers are increasingly turning to multi-agent systems
 - Multiple smaller, independent agents interacting within a network
 - Each agent can use simple direct LLM prompting or complex ReAct prompting (and more)
 - Collaborative goals
- Well-suited for scenarios where feedback from multiple personas is beneficial
 - Useful when parallelization across distinct tasks or workflows is required
 - Allowing individual agents to proceed with their next steps without being hindered by the state of tasks handled by others
- Benefits: modularity (easier to develop, test, and maintain), specialization (expert agents focused on specific domains), inter-agent communication control

Coordinating MAS

- Modern Agentic AI employs **Prompt-Driven Orchestration** to coordinate the system behavior without programming
 - Such coordination emerges stochastically from the exchange of natural-language messages between agents
- **The Orchestrator:** at the center sits a “Router” (often an LLM itself)
 - Instead of executing a conditional loop, it **reads** the current context and **generates a decision** on which specialized agent should speak or act next.
- **The Mechanism:** relies on structured conversation – agents “act” by generating text that **triggers** API calls or prompts other agents
 - The “state” of the program is the history of the conversation, maintained within the LLM's context window



Agents coordination patterns & methodologies

- While the **mechanism** is *conversational*, the **structure** of such a conversation dictates the agent's capability

Unstructured Collaboration – The “Chat Room”

Agents (e.g., a "Coder" and a "User Proxy") interact in a shared environment where the conversation loop is the workflow. The LLM acts as a dynamic router, deciding who speaks next based on the chat history.

+ This topology is ideal for **exploratory tasks** where the solution path isn't known in advance.

Hierarchical Orchestration – The “Corp Chat”

A “**Manager**” agent breaks down a high-level objective and assigns specific sub-goals to “**Worker**” agents. Unlike the chat room, this limits the conversation to specific channels, creating a **deterministic pipeline**.

+ Best suited for complex analytical tasks (like market analysis) that require a clear, auditable trail of actions.

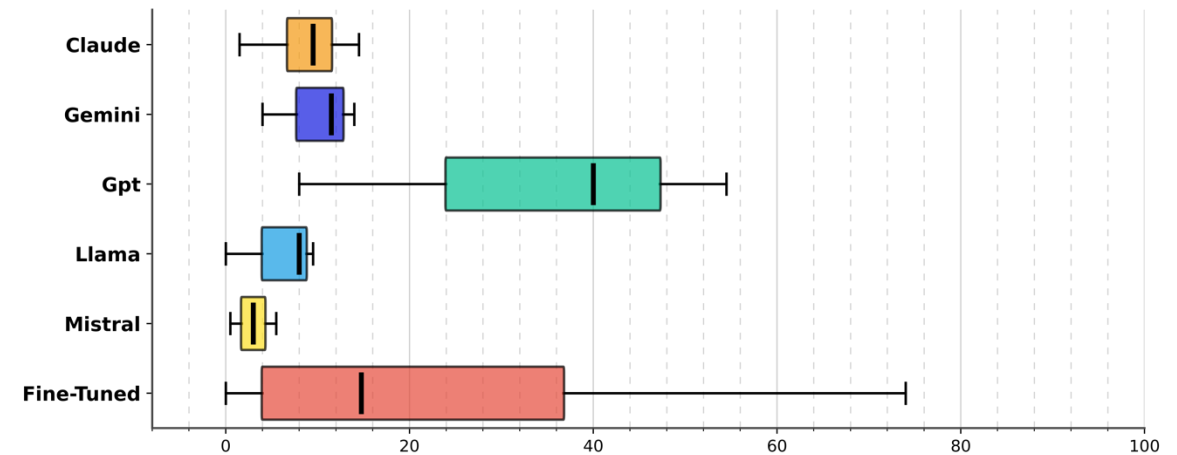
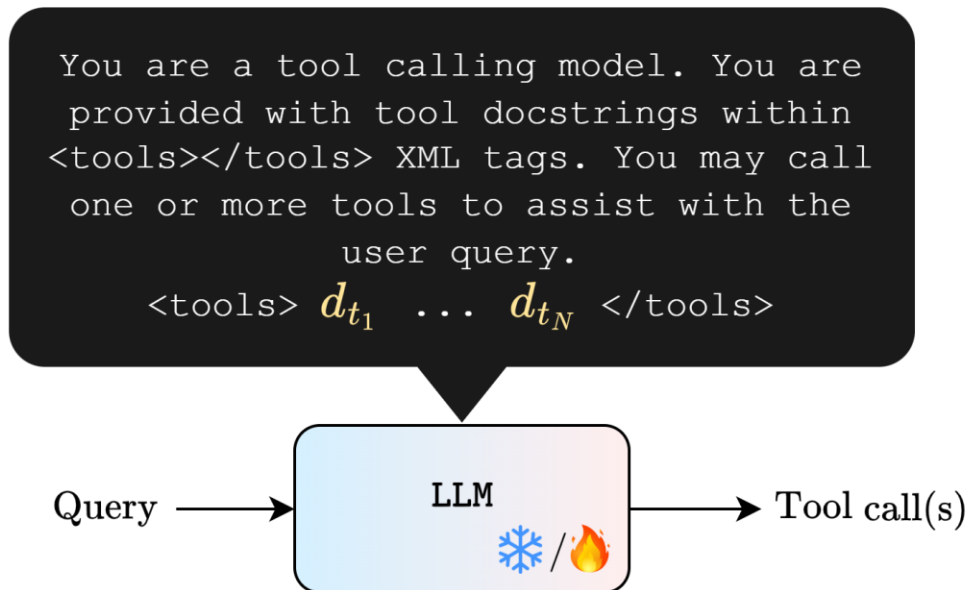
State-Flow Control – The “Graph”

Agents are nodes of a **state machine**. The generated content remains stochastic, but graph edges impose the flow. This allows for **forcing loops** (e.g., "If the code fails the test, loop back to the Coder").

+ Essential for production applications where reliability is more important than creativity.

Tool Learning Methods Struggle to Scale Up

- Common approaches for tool learning fall into two categories
 - 🔥 **Fine-tuning.** Expensive and confines knowledge to a predefined toolbox.
 - ❄️ **In-context learning.** Tool docstrings d_t with demonstrations to describe intended use. When tools are many, all their docstrings may not fit in the prompt—or, if they do, finding the right one becomes a needle-in-a-haystack problem.



Tool calling accuracy for LLMs degrades in lengthy and information-dense contexts [[BFCL V3 Long-Context](#)]

Toolformer: LLMs can teach themselves to use tools

- While tool calling is often an emerging capability of pretrained LLM, **finetuning** on a specific set of *private* tools is an option
- **Toolformer** ([T. Schick et al., 2023](#)) introduces a **self-supervised** approach to discover which API calls aid in predicting future tokens
 - Without any human-annotated dataset
 - This allows the model to maintain its general generative abilities while selectively using external tools to reduce perplexity **when needed**

The New England Journal of Medicine is a registered trademark of [QA("Who is the publisher of The New England Journal of Medicine?") → Massachusetts Medical Society] the MMS.

Out of 1400 participants, 400 (or [Calculator(400 / 1400) → 0.29] 29%) passed the test.

The name derives from "la tortuga", the Spanish word for [MT("tortuga") → turtle] turtle.

The Brown Act is California's law [WikiSearch("Brown Act") → The Ralph M. Brown Act is an act of the California State Legislature that guarantees the public's right to attend and participate in meetings of local legislative bodies.] that requires legislative bodies, like city councils, to hold their meetings open to the public.

There is **no preemptive action planning**

Tools are called when the model *feels* the need for them – to lower uncertainty

Training LLM for Tool Calling... *without supervision*

1 Sampling and Execution Phase

The training process begins with a “noisy” exploration phase – the LLM is prompted to sample potential API calls c_i at various positions within the text

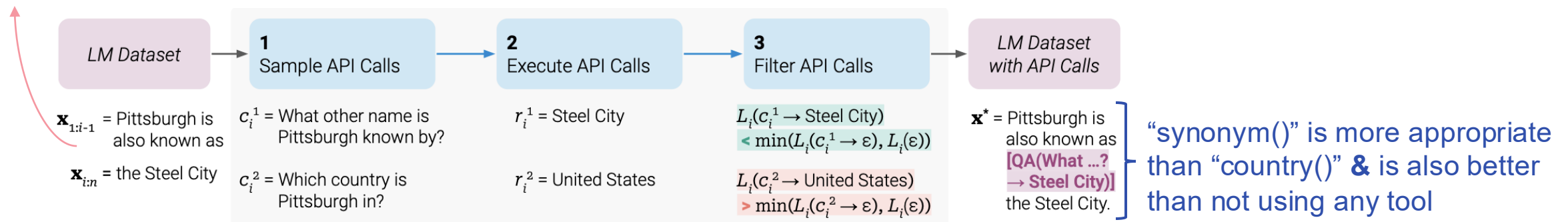
2 Filtering: Utility as Loss Reduction

Compare the *weighted cross-entropy* loss of predicting the subsequent tokens

- **with** the API call and its result r_i provided
- **without** the tool – simply continuing the generation using internal knowledge

An API call is retained in the fine-tuning dataset *only if* using it results in a **reduction** of the loss function
→ the tool **provides information** that the model could not easily predict on its own

Interleave tool calls within the text



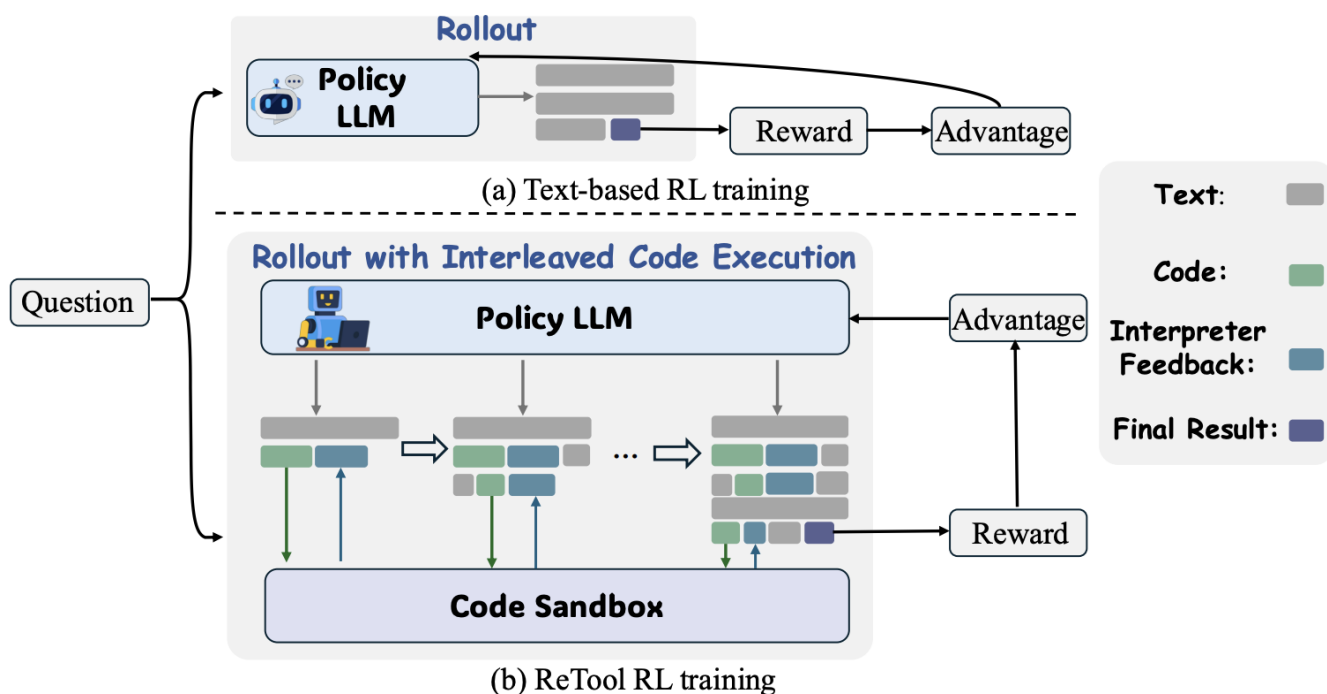
Reinforcement Learning for tool calling

- The “*do as I do*” training paradigm, typical of supervised settings, lacks sufficient trajectory **exploration** – essential for planning in complex domains
- In this *imitation game*, the model is trained to maximize next token prediction accuracy, or lower perplexity, **without dependence** between API invocations
 - The LLM learns to invoke tools but misses the understanding of the general goal
- In a RL setting, instead, the LLM can optimize trajectories **toward** a specific **reward** that measures the quality of the final output
 - Optimization for a **scalar score** (e.g., answer completeness, total number of used tools)
 - The agent can discover **novel strategies** that were not in the training data

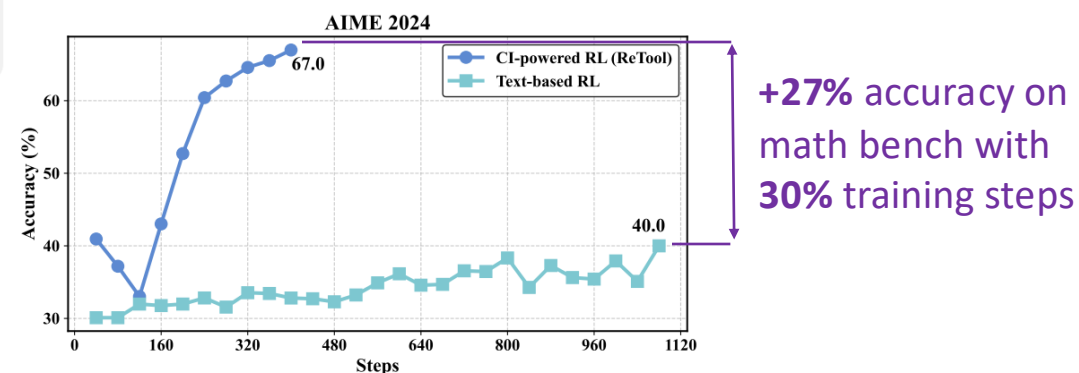
RL can be considered a completely autonomous system that depends on the soundness and quality of reward functions. In these settings, the definition of correct objectives is crucial for obtaining the desired agent behavior – as important as choosing the correct loss function in supervised settings

ReTool: RL for Strategic Tool Use in LLMs

- **ReTool** ([J. Feng et al., 2025](#)) introduces a RL framework that supports **dynamic**, multi-turn **rollouts** where the model can pause generation to execute code in a **sandbox** (Code Interpreter **CI**) and receive immediate feedback
 - Based on the [PPO](#) algorithm, the training employs a -1/1 correctness reward for the output



- 1 Use of a prompt template to guide the model in interacting with the sandbox by utilizing tags `<code></code>`
- 2 During the rollout process, the policy model generates text-based reasoning after the code termination trigger (`</code>`)
- 3 Upon completion, the sandbox's output is provided back to the model to continue generation



DR-Tulu: RL with evolving rubrics for Deep Research

- **Deep-Research** is a complex task for RL settings because there is no clear reward function to score the “quality” of a literature analysis
 - Quality is **subjective** (e.g., exhaustive? goal-centered? official references?)
- DR-Tulu ([R. Shao et al., 2025](#)) introduces **RL with Evolving Rubrics (RLER)** to **adapt** the **verifier** to the policy model during training



User

How did Netflix manage to successfully adapt One Hundred Years of Solitude, a notoriously difficult book to bring to the screen?



DR Tulu

Agentic Workflow

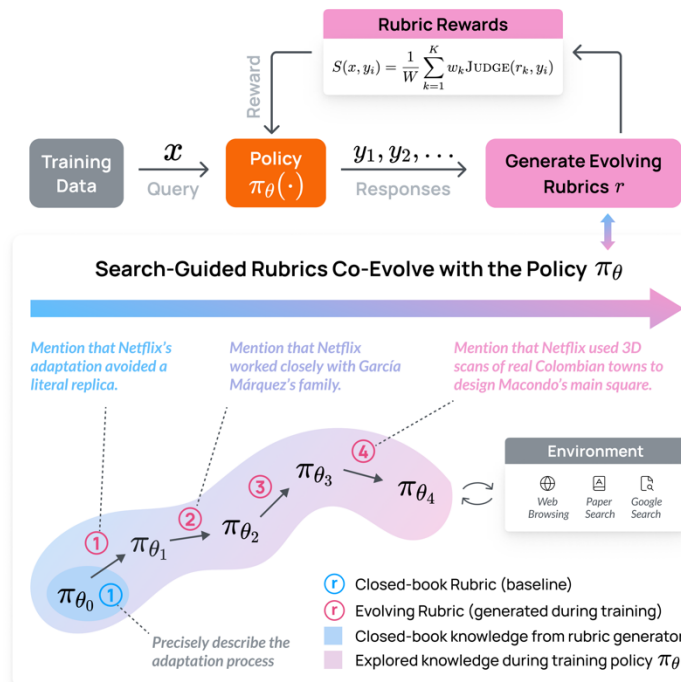
Think Tool 1 Think Tool 2 ... Answer

Long-form Report with Citations

Netflix's adaptation **avoided a literal replica of Macondo** and instead fused real locations with **meticulously built sets** to honor the novel's essence while giving the show contemporary visual grammar. **The production grounded magical realism in front-of-camera practical craft, relying on makeup, special effects,** ... The location strategy and production design...

Sources

- [1] The production team behind Netflix's adaptation of "One Hundred Years of Solitude," LA Times
- [2] The article discusses Netflix's adaptation of Gabriel García Márquez's celebrated novel, NY Times



Standard RL settings (e.g., GRPO) employ pre-defined **static** rewards

- Not suitable for long context tasks



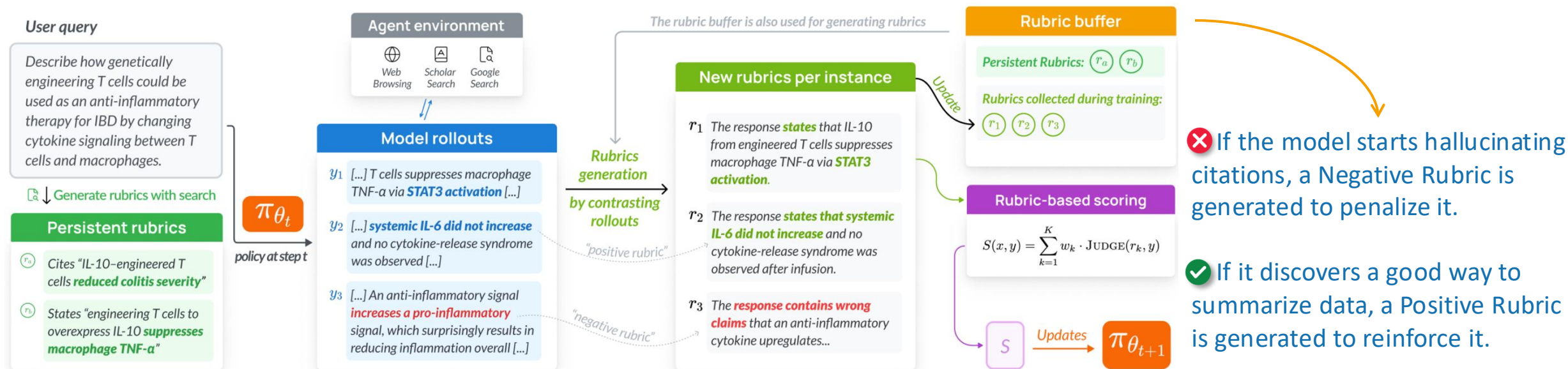
Using **LLM-as-a-judge** for scoring alleviates staticity but suffers from “reward hacking” and closed parametric knowledge



None of these methods adapts as the policy explores new evidence or behaviors.

Training with RLER

- To solve the *verifier problem*, RLER creates a **dynamic grading system** – as the agent gets smarter, the test gets harder
 - 1 Generate **persistent rubrics** to serve as an anchor during training, ensuring the model never drifts from fundamental requirements like factual accuracy, relevance, and formatting
 - 2 At each training step, generate temporary **evolving rubrics** by contrasting the model's current best and worst rollouts to penalize emerging failures or "reward hacking" strategies
 - 3 The policy is updated using a **composite reward signal** derived from both rubric sets

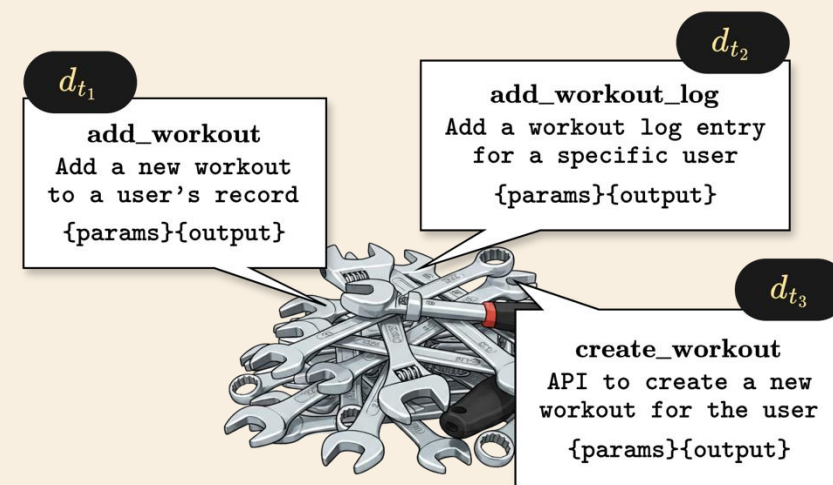
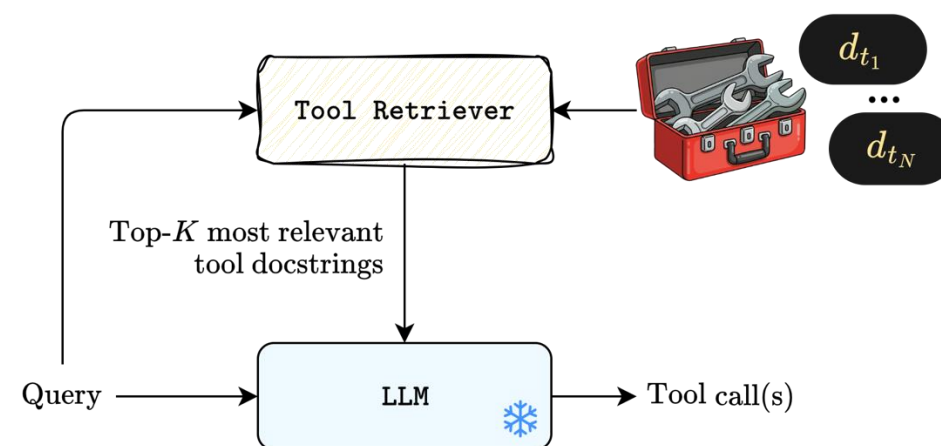
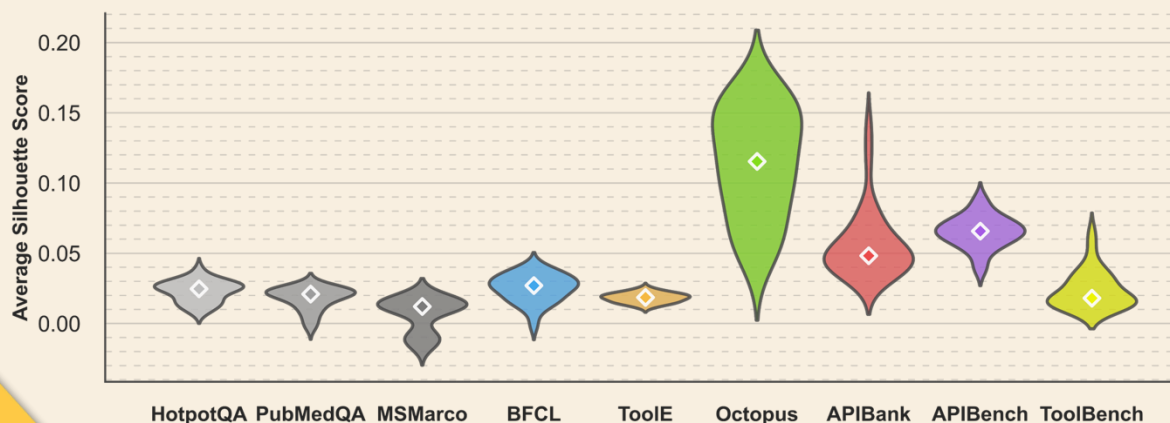


Tool Retrieval: Need for Optimization

- Retrieval-based methods to **preselect the most relevant tool docstrings**

Vanilla SOTA encoders struggle to select right tools.

Without fine-tuning, docstring embeddings are highly concentrated in space—recurring data types, keywords, and concise but distinctive signatures. Docstrings similar to the user query may be irrelevant.



Outline

- The illusion of Reasoning: Pitfalls of “vanilla” LLMs
- The Architecture of Autonomy: ReACT
- Large Action Models and Tool Calling
- **Model Context Protocol** 
- Examples of Application

The need for standardization



Without a standard for **tools format**, systems default to “Context Stuffing”, that is pushing massive, **static** JSON dumps into the prompt at inference time

- increased **latency** due to heavy prompt processing, and degraded reasoning performance
- If the provider **changes** their API (e.g., renaming a field), the LLM has no way of knowing

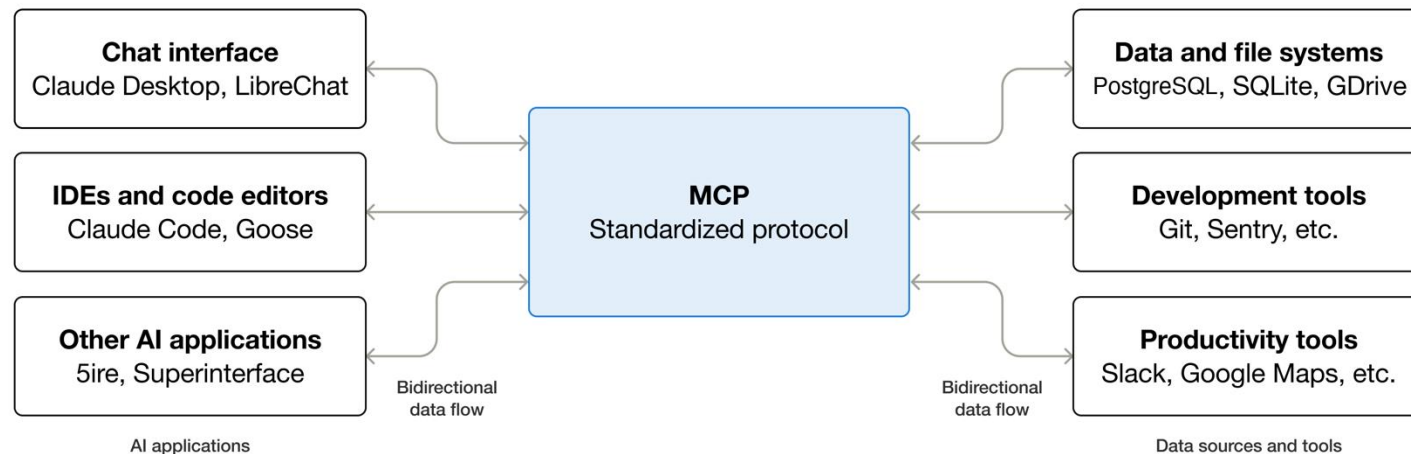


Every integration between an AI application and a data source requires custom, bespoke **connectors**, creating an exponential “ $N \times M$ integration problem”

- build $N \times M$ separate integrations, with **fragmented** implementations and duplicated effort
- hard to **scale** truly connected systems and stifling innovation through high integration costs
- each integration handles authentication, data access, and prompt logic differently, creating unpredictable results and potential **security vulnerabilities**

Model Context Protocol (MCP)

- MCP, introduced by *Anthropic*, is a framework designed to standardize the interaction between LLMs and external tools
 - defines fundamental building blocks that **servers** present to **clients**
 - each **Tool exposes** name, description, and JSON Schema specifying its inputs and outputs (allows dynamic changes)
 - **Resources** are application-controlled, read-only data sources accessed via URIs and work similarly to REST GET endpoints
 - MCP is built on **JSON-RPC 2.0** as its messaging protocol



MCP for standardized tool interfaces

- MCP follows a **three-tier architecture** where each component has distinct responsibilities and end-points



Host: core application (e.g., Claude Desktop) that manages multiple client instances, controls **permissions**, enforces security policies, and **coordinates** the LLM **integration**



Client: maintains a dedicated, stateful 1:1 connection with a single server, **routing** JSON-RPC messages bidirectionally while handling capability **negotiation** and ensuring security isolation between different servers



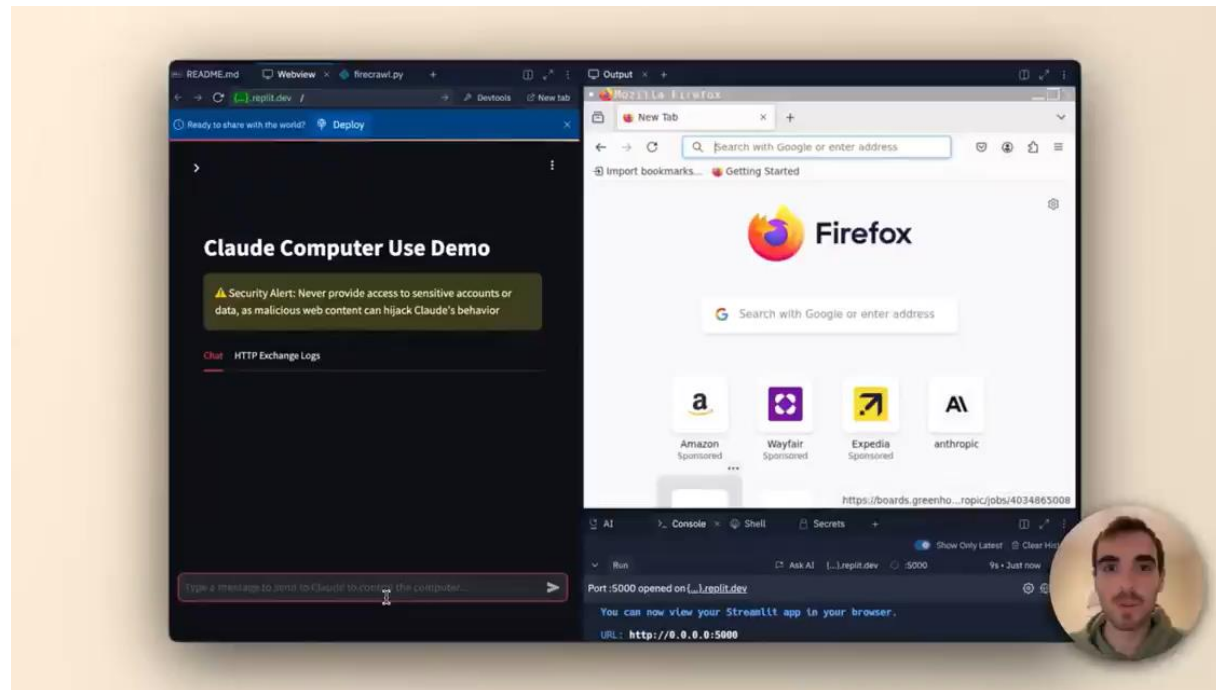
Server: capability **providers** that expose tools, resources, and prompts through the MCP primitives. It operates **independently** with **focused responsibilities** and can run either locally using STDIO transport or remotely using HTTP with Server-Sent Events (SSE)

Outline

- The illusion of Reasoning: Pitfalls of “vanilla” LLMs
- The Architecture of Autonomy: ReACT
- Large Action Models and Tool Calling
- Model Context Protocol
- **Examples of Application** 

Anthropic Computer Use API

- Developers can direct Claude to use computers the way people do—by **looking at a screen, moving a cursor, clicking buttons, and typing text**
 - Claude 3.5 Sonnet is the first frontier AI model to offer computer use in public beta
 - The video demonstrates how Computer Use API uses firecrawl API to gather information from web pages and then **fill a job application**

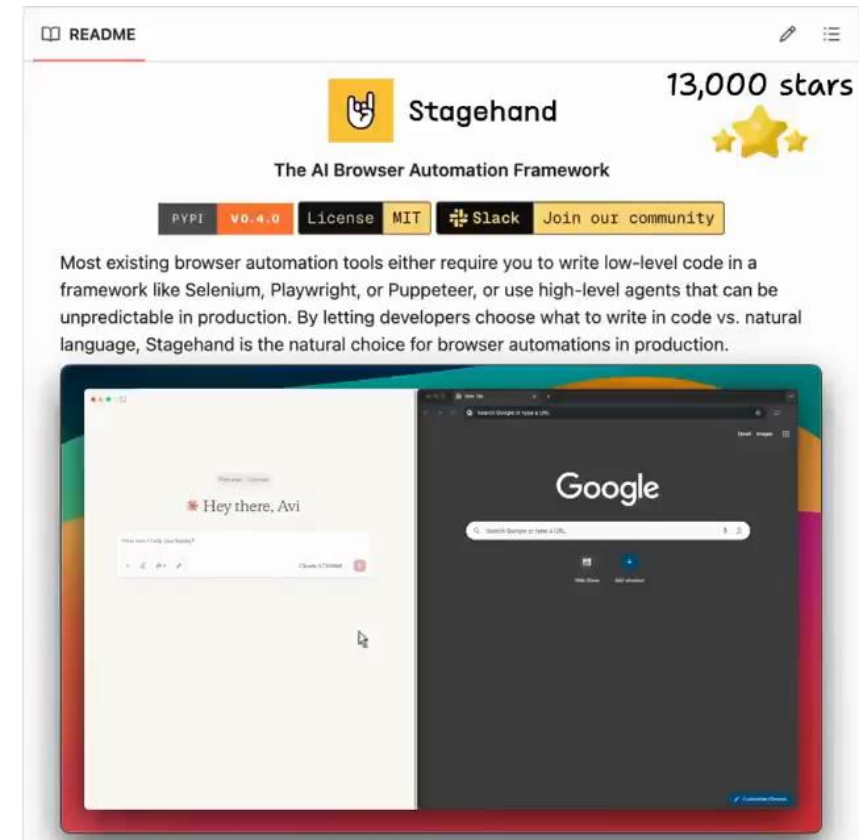


Browser automation

- **Stagehand** (by Browserbase) is an AI browser automation framework bridging the gap between brittle automation and unpredictable full-agent solutions

- ❌ Traditional browser automation frameworks (e.g., Playwright, Selenium) crumble when a website changes just slightly. A tiny UI tweak can break an entire script
- ❌ Full agent-based solutions allow full automation from just a prompt, but such an abstraction yields unpredictable outcomes

	Legacy frameworks	Stagehand	Browser agent
Core features			
Can read page content	✓	✓	✓
Can interact with page	✓	✓	✓
Continues to work when the page changes	—	✓	✓
Extract data using natural language	—	✓	✓
Follows the same steps each time	✓	✓	—
Reliable	✓	✓	—



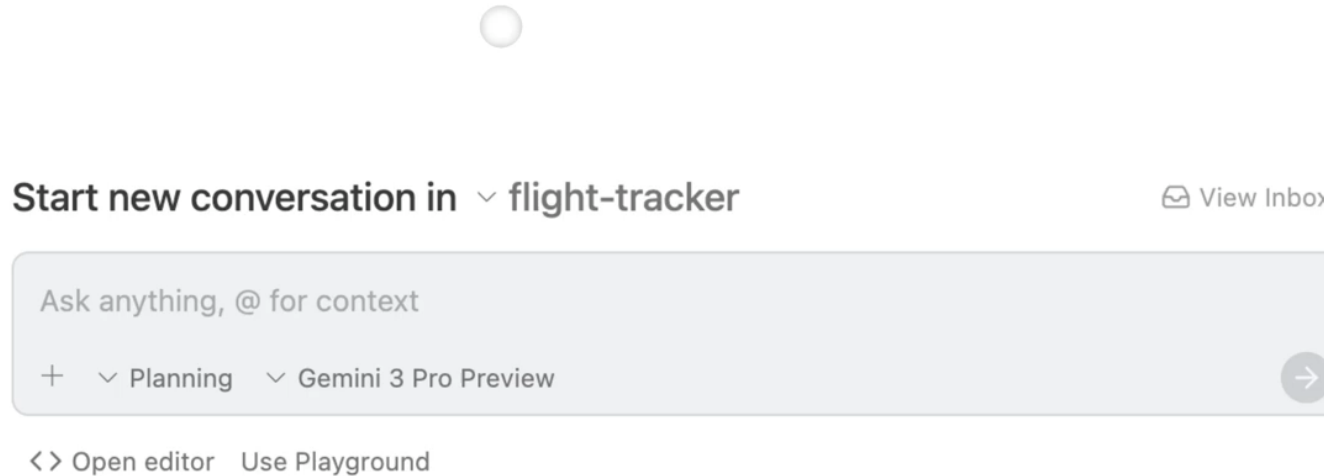
Claude Skills

- Allows users to package domain-specific expertise, workflows, and instructions into **reusable folders** that can dynamically be loaded when relevant to a task
 - At startup, the agent pre-loads only the name and description of every installed skill into its system prompt
 - When Claude detects a relevant task, it loads the full instructions, scripts, and resources from that skill folder.



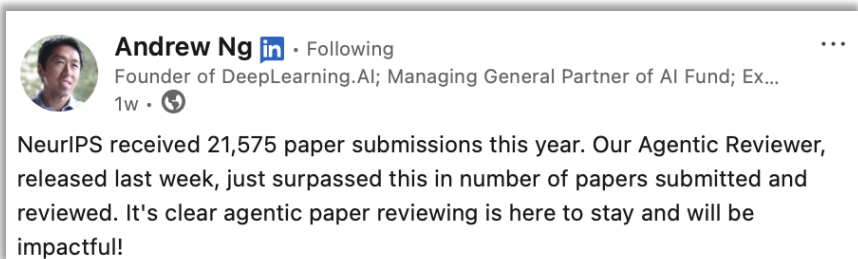
Google Antigravity

- Along with Gemini 3, Google has just released its own **agentic IDE** capable of interacting, running, and testing code on external environments like browsers.



Stanford Agentic Reviewer

- paperreview.ai provides feedback on research papers developed by Stanford
 - 1 Generates targeted search queries to **retrieve** the most relevant prior work from arXiv
 - 2 Creates **summaries** of related research
 - 3 It **synthesizes** these artifacts to generate a comprehensive, grounded review



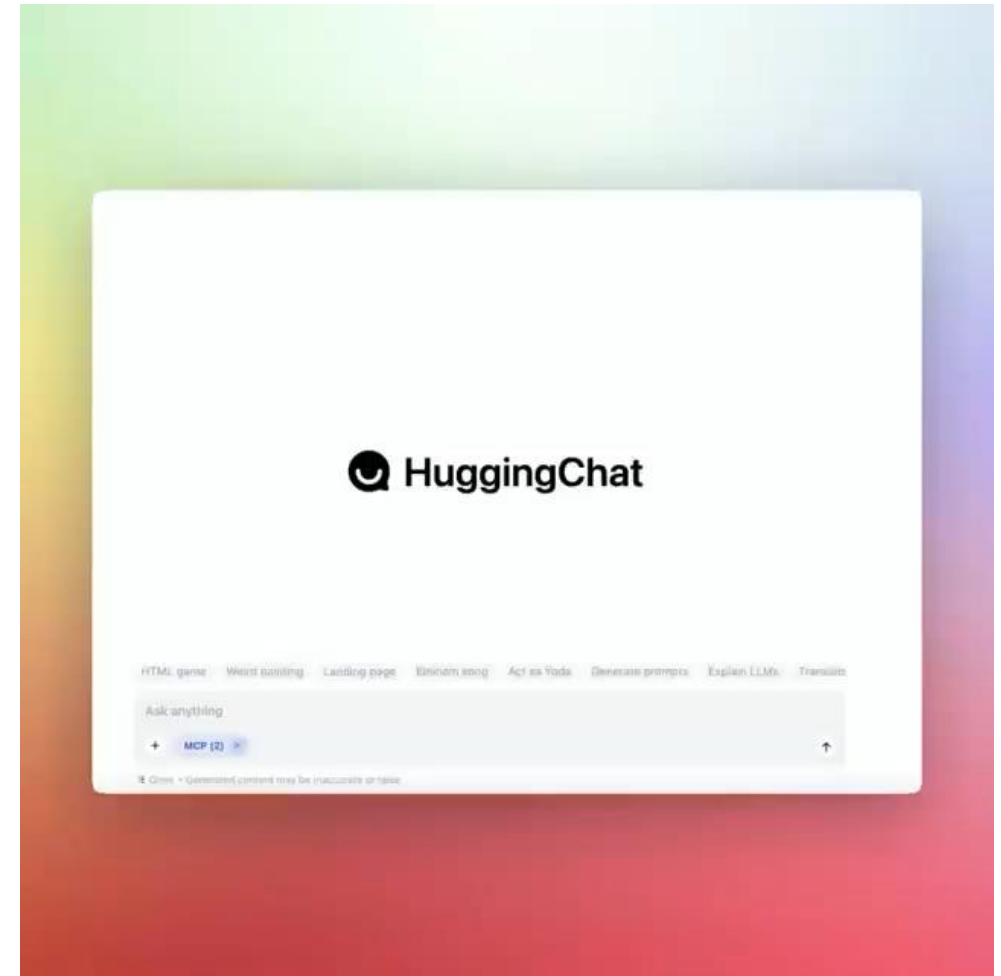
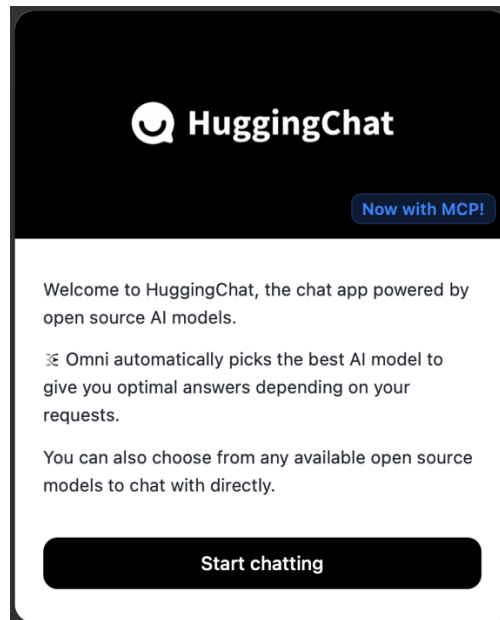
When we trained the system on ICLR 2025 reviews and measured Spearman correlation (higher is better) on the test set:

- Correlation between two human reviewers: 0.41
- Correlation between AI and a human reviewer: 0.42

A screenshot of the paperreview.ai submission interface. At the top, a "How it works" section shows three steps: 1. Upload & Submit (Upload your paper PDF and enter your email address), 2. Get Notified (Receive an email notification when the AI review is complete), and 3. View Review (Come back to see the AI review and suggestions for your work). Below this is a "Paper PDF *" section with a large drop zone containing a document icon and the text "Choose PDF file or drag and drop" and "Max 10MB • First 15 pages analyzed". Underneath is an "Email Address *" field with a placeholder "your.email@university.edu". Then there is a "Target Venue (Optional)" dropdown menu with the text "Select a conference/journal". At the bottom is a large red button labeled "Submit for Review".

HuggingChat w/ MCP

- **Hugging** Face's open-source **chat** interface now supports MCP, enabling LLMs to access
 - Models and datasets
 - Spaces and papers
 - Run community tools such as Gradio apps hosted on Spaces.



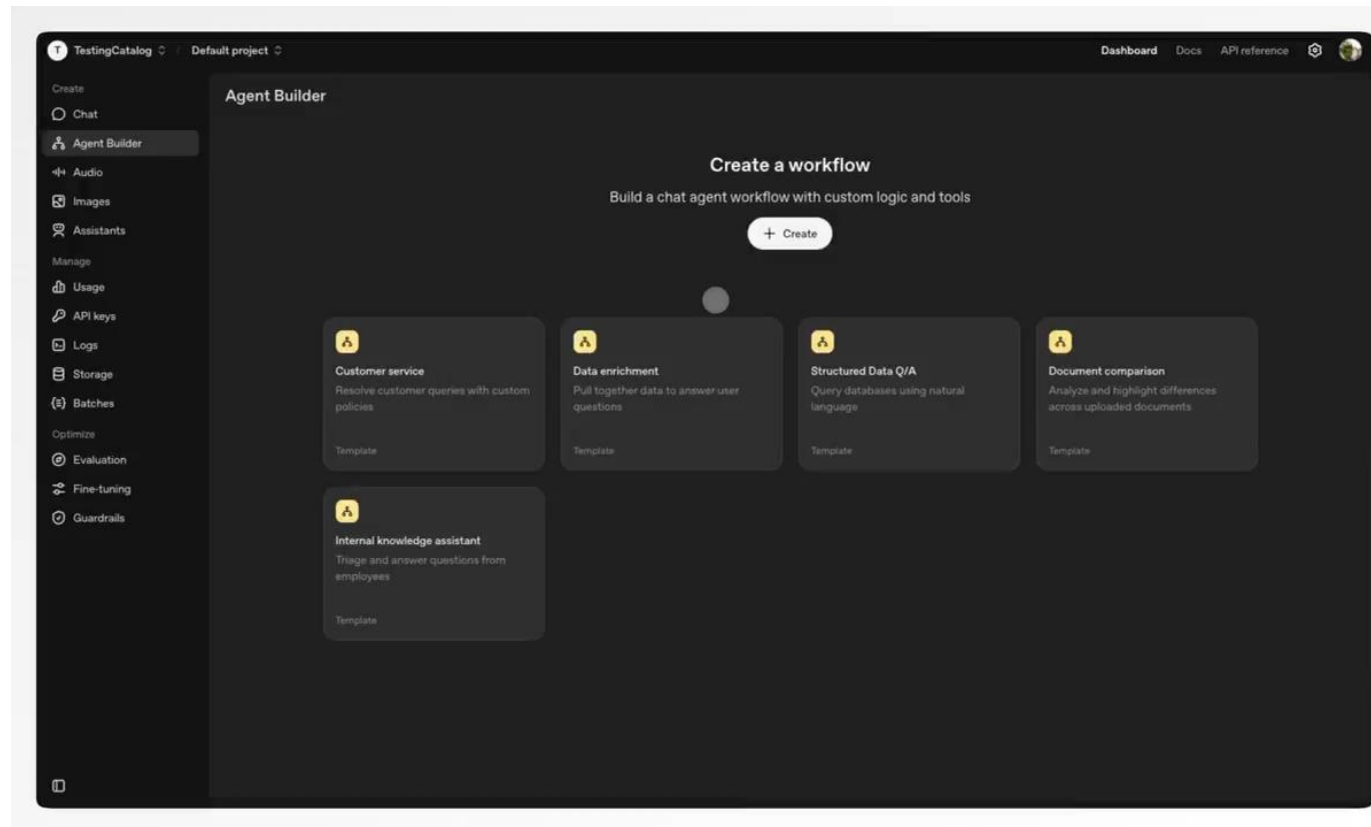
LightAgent for edge devices

- Lightweight **mobile agent framework** that enables true on-device deployment of GUI agents on smartphones through compact 3B-scale Vision-Language Models and intelligent device-cloud collaboration



OpenAI Agent Builder

- A visual, **no-code** platform for building production-ready AI agents.
- Introduces a **drag-and-drop canvas** that allows users to create agent flows from predefined templates





... for the column “What happened last night?”

MCP for everyone...

“Donating the Model Context Protocol and establishing the Agentic AI Foundation”



Donating the Model Context Protocol

Anthropic is donating the Model Context Protocol to the Linux Foundation's new Agentic AI Foundation, where it will join goose by Block and AGENTS.md by OpenAI as founding projects. Bringing these and future projects under the AAIF will foster innovation across the agentic AI ecosystem and ensure these foundational technologies remain neutral, open, and community-driven.

The Model Context Protocol's governance model will remain unchanged: the project's maintainers will continue to prioritize community input and transparent decision-making.

AI Anthropic @AnthropicAI

Anthropic is donating the Model Context Protocol to the Agentic AI Foundation, a directed fund under the Linux Foundation.

In one year, MCP has become a foundational protocol for agentic AI. Joining AAIF ensures MCP remains open and community-driven.

[Traduci post](#)

Donating the Model Context Protocol and establishing the Agentic AI Foundation

Da anthropic.com

6:01 PM · 9 dic 2025 · 294.320 visualizzazioni

... Agents for everyone indeed

OpenAI also participates in this *donation* with their AGENTS.md

With AGENTS.md, we introduced a lightweight Markdown file designed to live alongside README.md. It offers a simple, interoperable standard that gives AI coding agents a consistent source of project-specific guidance needed to operate reliably across different repositories and toolchains. This markdown-based convention makes agent behavior far more predictable

across diverse repositories and build systems.

Sample AGENTS.md file

Dev environment tips

- Use ``pnpm dlx turbo run where <project_name>`` to jump to a package instead of scanning with ``ls``.
- Run ``pnpm install --filter <project_name>`` to add the package to your workspace so Vite, ESLint, and TypeScript can see it.
- Use ``pnpm create vite@latest <project_name> -- --template react-ts`` to spin up a new React + Vite package with TypeScript checks
- Check the name field inside each package's package.json to confirm the right name-skip the top-level one.

Testing instructions

- Find the CI plan in the .github/workflows folder.
- Run ``pnpm turbo run test --filter <project_name>`` to run every check defined for that package.
- From the package root you can just call ``pnpm test``. The commit should pass all tests before you merge.
- To focus on one step, add the Vitest pattern: ``pnpm vitest run -t "<test name>"``.
- Fix any test or type errors until the whole suite is green.
- After moving files or changing imports, run ``pnpm lint --filter <project_name>`` to be sure ESLint and TypeScript rules still pass
- Add or update tests for the code you change, even if nobody asked.

PR instructions

- Title format: [`<project_name>`] `<Title>`
- Always run ``pnpm lint`` and ``pnpm test`` before committing.



Updates for coding agents

Google Cloud Tech @GoogleCloudTech

Introducing AlphaEvolve, our Gemini-powered coding agent for designing advanced algorithms—in private preview.

AlphaEvolve uses LLMs to propose intelligent code modifications, creating a feedback loop that evolves your algorithms to be more efficient → goo.gl/4puV5GZ

[Traduci post](#)

```
parent_program, inspirations = database.sample()
prompt = prompt_sampler.build(parent_program, inspirations)
diff = llm.generate(prompt)
child_program = apply_diff(parent_program, diff)
results = evaluator.execute(child_program)
database.add(child_program, results)
```

AlphaEvolve

10:13 PM · 9 dic 2025 · 6.005 visualizzazioni

Claude @claudeai

We've shipped three new updates for Claude Agent SDK to make it easier to build custom agents:

- Support for 1M context windows
- Sandboxing
- V2 of our TypeScript interface

[Traduci post](#)

```
Agent SDK developers!

Users can now use the Agent SDK
with the 1M context window version
of Claude Sonnet 4.5.
```

6:35 PM · 9 dic 2025 · 54.646 visualizzazioni

And after the prompt engineer...

Langchain released the blog “[Agent Engineering: A New Discipline](#)”

What is agent engineering?

Agent engineering is the iterative process of refining non-deterministic LLM systems into reliable production experiences. It is a cyclical process: **build, test, ship, observe, refine, repeat.**

Agent engineering is emerging because the opportunity demands it. Agents can now handle workflows that previously required human judgment, but **only if you can make them reliable enough to trust.** There is no shortcut, just the systematic work of iteration. The question isn't whether agent engineering will become standard practice. It's how quickly your team can adopt it to unlock what agents can do.

 **LangChain** 
@LangChainAI

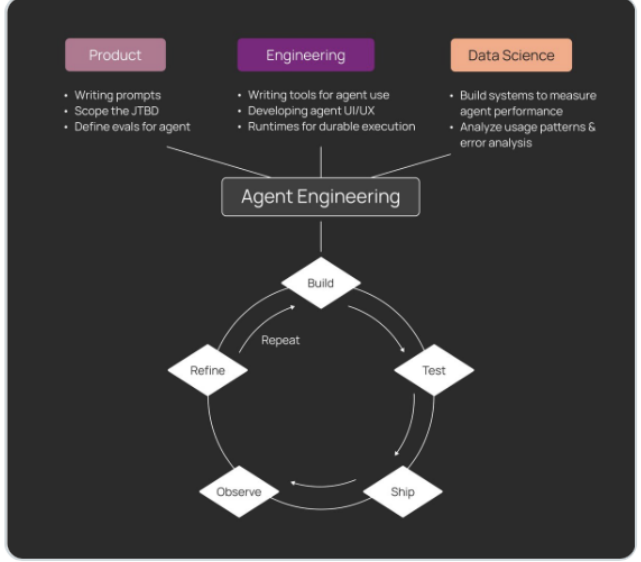
Agent engineering: A new discipline

Traditional software assumes known inputs and predictable behavior. Agents give you neither. That's why teams shipping reliable agents are adopting a new discipline: agent engineering.

Agent engineering is driven by a few core ideas:

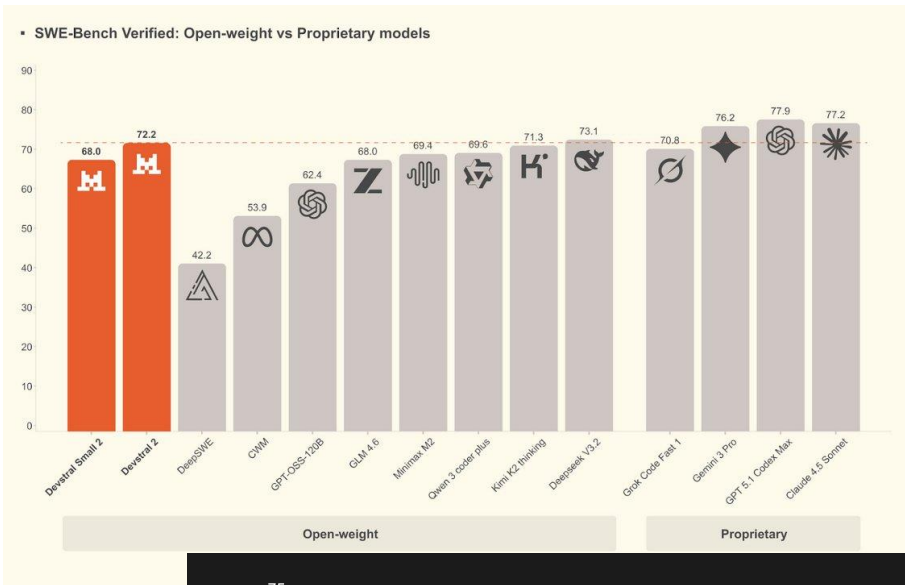
- ◆ Combine engineering, product, and data science skills to shape agent behavior
- ◆ Real usage in production shows what actually breaks and where assumptions fall apart
- ◆ Agent quality comes from continuous iteration: i.e. ship, observe, refine prompts and tools, repeat

Read to learn more about why agent engineering is emerging now — and what it looks like in practice: blog.langchain.com/agent-engineer...
[Traduci post](#)



7:24 PM · 9 dic 2025 · 13.538 visualizzazioni

Mistral AI releases **Devstral-2**





Mistral AI

@MistralAI

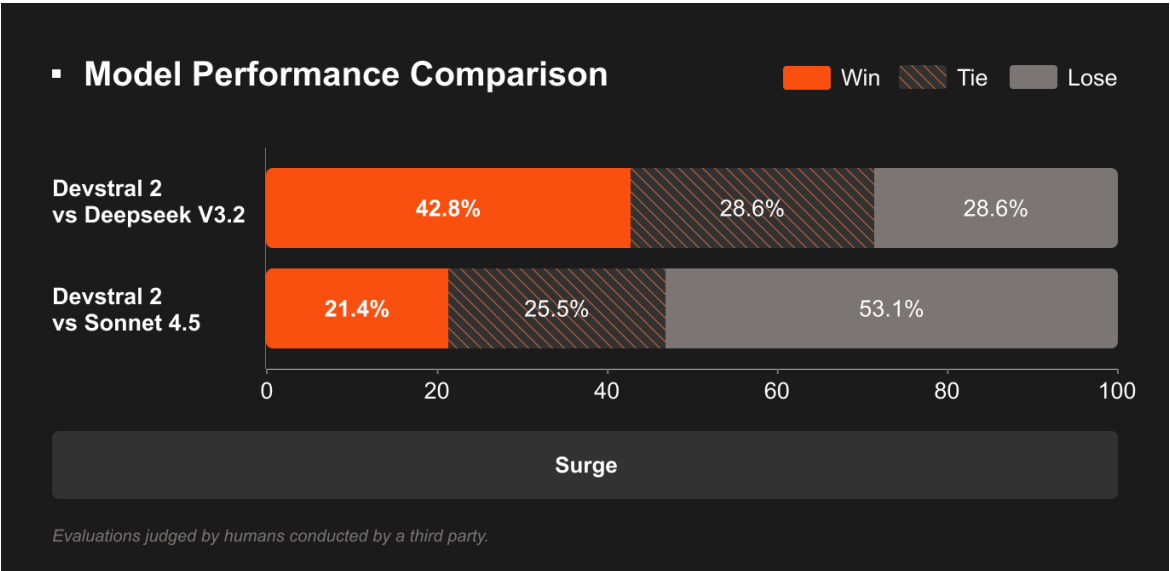
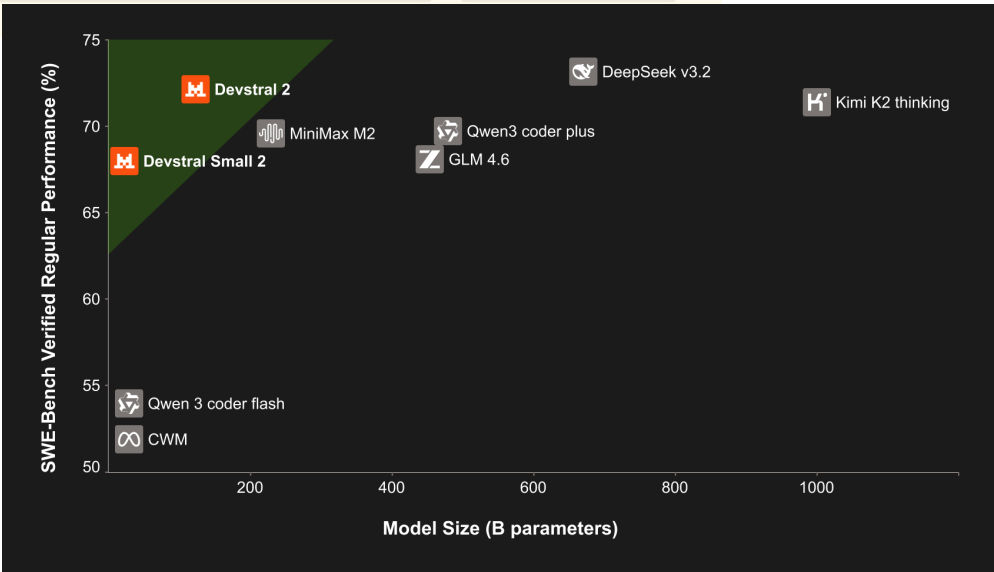
Introducing the Devstral 2 coding model family. Two sizes, both open source.

Also, meet Mistral Vibe, a native CLI, enabling end-to-end automation.

 Traduci post

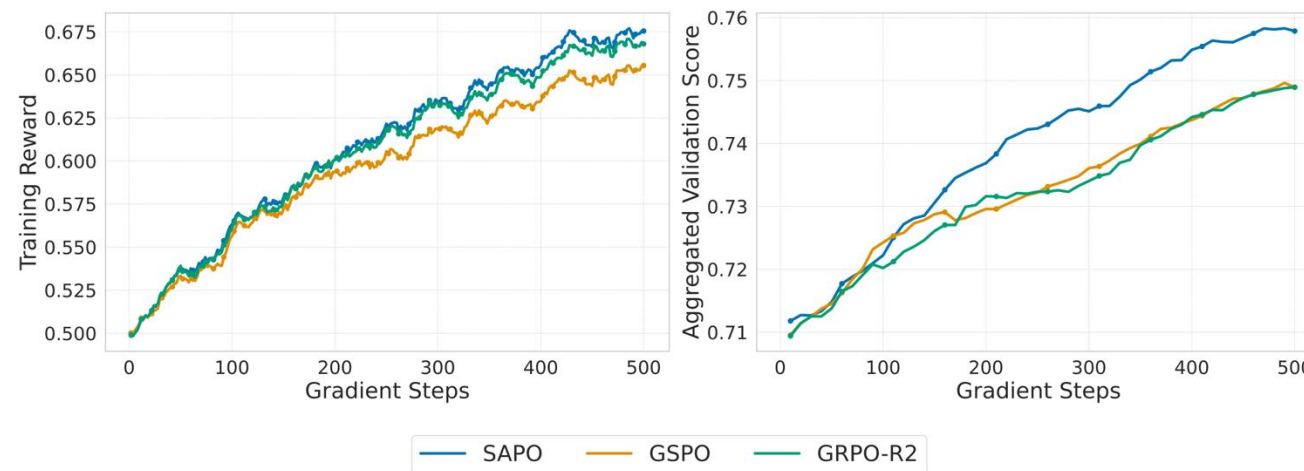
4:00 PM · 9 dic 2025

· 454.151 visualizzazioni



GRPO: another variant, again...

- SAPO ([C. Gao et al., 2025](#)) addresses the problems related to hard clipping and the discard of useful learning signals during training
- SAPO is better sample efficient, while improving training stability



A brief critical note about GRPO

Can we really consider GRPO an RL framework? Can real exploration be done within-distribution? Or are we simply trading pass@N with pass@1?

 **Qwen**
@Alibaba_Qwen

🚀 We introduce Soft Adaptive Policy Optimization (SAPO) — a smooth, stable, and highly effective RL method for training large language models.

Why SAPO?

- ♦ Hard clipping is brittle — gradients vanish or explode
- ♦ MoE models amplify variance, making training even more unstable

SAPO replaces hard boundaries with a continuous, temperature-controlled gate that:

- ★ Smooth trust-region behavior → no abrupt gradient drop
- ★ Sequence-level coherence → align sequence-level behavior
- ★ Token-level adaptivity → preserves useful gradients & boosts sample efficiency
- ★ Asymmetric temperatures → significantly improved stability, esp. in MoE models

What does this mean in practice?

- ✓ Longer stable RL runs
- ✓ Higher Pass@1
- ✓ Stronger performance on Qwen3-VL across math, coding & multimodal tasks

SAPO offers a more scalable and reliable foundation for RL-tuning large language & multimodal models.

📄 Paper: arxiv.org/abs/2511.20347
📝 Blog: qwen.ai/blog?id=sapo
Traduci post



arxiv.org
Soft Adaptive Policy Optimization
Reinforcement learning (RL) plays an increasingly important role in enhancing the reasoning capabilities of large ...

8:55 AM · 9 dic 2025 · 63.177 visualizzazioni



Thanks for the attention.
Is all you need.